Capstone 03 – Day 1 & Day 2 Hands-On Guide

Flight Delay and Airport Operations · Day 1 = Bronze · Day 2 = Silver · Databricks Enterprise · working solo

0. Read this first

How to use this guide. Go top to bottom. Every step ends with an **Expected result**. If yours does not match, stop and fix it before moving on, because later steps depend on earlier numbers. Each code block is one notebook cell unless I say otherwise.

What I did before writing it. I opened your zip, profiled all 21 files, and then ran the Silver cleaning and CDC logic in this guide against your real CSVs in a local Spark session. Every number marked *Expected* comes from that run, not from guesses.

What I could not run. Auto Loader, Unity Catalog volumes and Delta `MERGE` only exist inside a Databricks workspace, so I could not execute those parts. The code follows documented behaviour, but you are the first to run it. Section 12 lists the failures I would expect and how to fix each.

Corrections to the earlier plan

After reading the real files, several things in the earlier plan were wrong or incomplete. Where they differ, use this guide.

Earlier plan said	What your data actually shows	What it changes
Extreme values are in `fare_amount` and `delay_minutes`	All 35 are in `fare_amount` (8,110 to 77,083 against a normal 100 to 1,100). `delay_minutes` only runs 0 to 90	The outlier rule is on fare only
Cancellation rate comes from `flight_status`	No `CANCELLED` exists in the flight events (only OPEN, PENDING, COMPLETED, FAILED). It appears only in the status updates (171 flights)	Cancellation rate must use the status *after* CDC (Day 3)

Earlier plan said	What your data actually shows	What it changes
Dedupe using ingest timestamp as tie-breaker	Duplicates are exact copies with the same ingest timestamp, usually in different files. Copies in batches 10 to 12 carry an extra column	Hash the 12 base columns; break ties on file name
`timezone` may look like a currency	It is `USD` in all 11,540 rows	UTC conversion needs an explicit, documented assumption
Status CDC must handle late updates	The data has none: 520 updates, one per flight, all on 2026-08-10, perfectly in sequence	You have to create late and out-of-order cases yourself in tests
Temporal join on schedule revisions	There is no schedule data anywhere in the pack	Implement the rule on status revisions and say so openly

1. Your data, verified

Source	Rows	What I found
`primary_events` (12 files)	11,540	11,500 unique `flight_id`, so 40 exact duplicate rows (37 pairs sit in two different files, 3 inside one file). Batches 01 to 09 have 12 columns; batches 10 to 12 add a 13th, `source_schema_version` (value `v2`). Event times run 2026-07-01 to 2026-08-01, format `2026-07-17T07:46:00`
Missing values	12	All in `flight_type`, blank in the CSV
Extreme values	35	All in `fare_amount`: 8,110.50 to 77,083.50 on FLIGHT-0000301 to FLIGHT-0000335. Normal fares are 100.16 to 1,099.99
`flight_status`	–	OPEN, PENDING, COMPLETED, FAILED
`timezone`	–	`USD` in every row (a currency, not a time zone)

Source	Rows	What I found
`status_updates`	520	520 different flights, one update each. `change_timestamp` 2026-08-10 00:00 to 08:39, which is after every flight event. `source_sequence` 1 to 520. `new_status`: COMPLETED 176, REVERSED 173, CANCELLED 171. 45 rows are no-ops (previous = new). `previous_status` matches the flight's status in 100% of rows
`secondary_events`	310	All link to real flights. Types: CHECK 117, ALERT 97, REVIEW 96
`airline_id` / `airport_id`	110 / 150	Every key used in the flights exists in the lookups. Active flag Y/N: 104/6 and 142/8. `effective_date` is 2026-01-01 for every row

Total rows: 11,540 + 520 + 310 + 110 + 150 = **12,630**, which matches `rows_total` in `qa_manifest.json`.

2. How the work is laid out

```
landing/  (original files, never touched)
   |  copy_to_live()  <- lets you simulate files arriving over time
   v
live folders  ->  Auto Loader  ->  BRONZE (5 Delta tables, raw text + lineage
columns)
                                      |
                                      v
                                  SILVER (typed, deduplicated, rejected
rows, current state, status history)
                                      |
                                      v
                                  GOLD (Day 3)  ->  Dashboard (Day 5)
```

Why a `landing` folder and a live folder? On Day 4 you must show replay, schema drift and failure recovery. That needs files that *arrive* after the first run. Keeping originals in `landing` and copying them into the live folders on demand gives you that without re-uploading anything.

Notebooks you will create (all inside the `notebooks/` folder of your repo):

Notebook	Day	Purpose
`00_config`	1	Names, paths, Auto Loader helper
`01_profile_source`	1	Profile raw files, produce the facts for your decision log
`02_bronze_ingest`	1	Load five Bronze tables, show schema drift and replay safety
`03_silver_functions`	2	Cleaning, dedupe, CDC and as-of functions
`04_silver_events`	2	Build `silver_flight_events` and its rejects
`05_silver_cdc`	2	Build `silver_flight_current`, `silver_status_history`, apply CDC
`06_tests_silver`	2	Unit tests

Tables you will end up with: `bronze_flight_events`, `bronze_status_cdc`, `bronze_secondary_events`, `bronze_airline_id`, `bronze_airport_id`, `silver_flight_events`, `silver_flight_current`, `silver_status_history`, `silver_rejected_records`, plus one helper table `silver_flight_event_status_asof`.

3. Time budget

Day 1 (8 h)	Hours	Day 2 (8 h)	Hours
Workspace, Git, config, upload (section 4)	2.0	Rule catalogue and decisions (section 7)	1.0
Profiling and decision log (section 5)	2.0	Silver functions and events build (sections 8 to 9)	2.5
Bronze ingest, drift, replay (section 6)	2.5	CDC, history, UTC, as-of (section 10)	2.5
One-pager docs, commit, evidence (section 6.8)	1.5	Unit tests (section 11)	1.5
		Evidence pack and commit (section 13)	0.5

Rule: if you are more than 1 hour behind at the end of Day 1, drop the secondary events table and the HLD diagram polish. Never drop profiling, the Bronze audit, or the decision log.

3A. Before you start: notebook and Git basics

This section assumes nothing. Skim it once; come back when a step confuses you.

Notebooks

1. **Create one:** in the left sidebar open **Workspace**, go to your Git folder (or any folder), use **Create → Notebook**, and give it the name from the guide (for example `00_config`). The default language is Python.
2. **Attach compute:** top right of the notebook, pick **Serverless**, or an all-purpose cluster on a recent LTS runtime with Unity Catalog access and wait until it shows as connected.
3. **Cells:** a notebook is a stack of cells. Click a cell, paste the code, then press **Shift+Enter** to run it and move to the next cell. **Ctrl+Enter** (Cmd+Enter on Mac) runs it and stays put. To add a cell, hover between two cells and click **+ Code**.
4. **Output** appears under the cell. `print(...)` shows text. `display(df)` shows a table you can scroll and sort.
5. **`%run ./00_config`** runs another notebook from the same folder as if its code were pasted here. It must be alone in its cell.
6. **`%sql`** on the first line turns a cell into SQL.
7. **If a cell fails:** red text appears. Read the **last lines first**; they name the real problem. The first lines are usually just the call stack.
8. **Run all** (top toolbar) runs every cell in order. Use it at the end to prove the notebook works from top to bottom on its own.

Git: the click path

Labels move around between versions of the Databricks screens, so if a name differs slightly, look for the closest one.

1. **Create a GitHub token.** On GitHub: profile picture → **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**. Name it `databricks`, choose an expiry, under repository access pick **Only select repositories** and choose your repo, and under permissions set **Contents** to **Read and write**. Generate and copy it now; GitHub shows it once.
2. **Link it in Databricks.** Profile icon (top right) → **Settings** → **Linked accounts** (or **Git integration**) → provider **GitHub**, your username, paste the token, save.
3. **Create the Git folder. Workspace → Create → Git folder**, paste the repo URL, create.
4. **Make a branch.** Open the Git folder and click the branch name button, then **Create branch**, for example `feature/day1-bronze`.

5. **Commit and push.** In the same dialog, tick the changed files, type a message such as `day1: config notebook and decision log`, and choose **Commit and push**.
6. **Open a pull request.** On GitHub you will see a banner offering **Compare & pull request**. Click it, fill in the checklist below, create it, then **Merge**.

Pull-request checklist (save as `docs/pr_checklist.md` and paste it into each PR):

- [ ] Notebooks run top to bottom on fresh compute
- [ ] Expected counts from the guide were verified (list them)
- [ ] Decision log updated
- [ ] No tokens, passwords or personal data in any file
- [ ] Evidence saved under `docs/evidence/dayN/`
- [ ] README and limitations list updated

Keeping evidence

The spec asks for code, outputs, validation and limitations, *not screenshots alone*. For every check in this guide, do both: take a screenshot that shows the code **and** its output together, and paste the output as text into a markdown file in `docs/evidence/dayN/`. Name files so you can find them later, for example `day1_bronze_audit.png` and `day1_bronze_audit.md`.

4. Day 1 setup

Step 1.1 – Workspace and catalog

Before you write any code, find out what you are allowed to do. On enterprise workspaces most Day 1 delays are permissions, so check them first.

1. **Sign in and open a SQL cell** in a scratch notebook (**Workspace → Create → Notebook**, named `scratch`). Run `SHOW CATALOGS` and `SELECT current_user()`. Note which catalogs you can see.
2. **Choose a catalog.** Either one you have been given, or one you may create. You need `USE CATALOG` and `CREATE SCHEMA` on it (or `CREATE CATALOG` on the metastore if you want your own, for example `capstone03`). Use only letters, digits and underscores in the name.
3. **Choose compute.** Serverless notebooks if your workspace offers them; otherwise an all-purpose cluster on a recent LTS runtime with a Unity Catalog access mode (Standard or Dedicated). Every code cell in this guide is written to work on either.

4. **Choose where the files live (spec Day 1, task 4).** Preferred: an **external volume** on an *existing external location*, which is an ADLS Gen2 container your admin has registered. Find it under **Catalog → External data → External locations** and note its `abfss://...` URL. You need `CREATE EXTERNAL VOLUME` on that location plus `READ FILES` and `WRITE FILES`. Fallback: a **managed volume**, which needs only `CREATE VOLUME` on your schema. Record which one you used (D01).
5. **Choose how the reference data arrives.** Preferred: read the two lookup tables from a real Azure SQL Database over JDBC (section 6.5A). Fallback: the CSV extracts, labelled as a simulation. Record which one (D02).
6. **Check the Day 4 and Day 5 features now** so nothing surprises you later: can you create Workflows jobs with failure e-mail notifications, create a SQL warehouse, create an AI/BI dashboard, and see Genie? One line each in your RAID log is enough.

Missing a privilege? Send your admin this, with the names filled in:

> Hi, for a Databricks training capstone I need: USE CATALOG and CREATE SCHEMA on catalog `<name>`; CREATE EXTERNAL VOLUME, READ FILES and WRITE FILES on external location `<name>` (or CREATE VOLUME if no external location is available); permission to create jobs and a SQL warehouse; and, if possible, a secret scope for an Azure SQL connection. Thanks.

Tip: do not sit waiting for the admin. Start on Step 1.2 (GitHub) while you wait, and build the notebooks on a managed volume if that is all you can create today. Switching to an external volume later is a one-line change in `00_config`.

Step 1.2 – GitHub repo and Git folder

1. On GitHub create a repo, for example `capstone03-flight-ops`, with a README.
2. In Databricks: **Workspace → Create → Git folder**, paste the repo URL, choose GitHub. Databricks will ask for a GitHub personal access token with read and write access to that repo; create one in GitHub under Settings → Developer settings and paste it when prompted.
3. Inside the Git folder use the branch selector to create a branch named `feature/day1-bronze`. Work on this branch for Day 1.
4. Create three folders in the Git folder: `notebooks`, `tests`, `docs`.

If Git folders are not available in your workspace, develop in a normal workspace folder, and after each major step use **File → Export → Source file** and commit the files to GitHub by hand. Write that down as a limitation. The spec grades Git *evidence*, not the mechanism.

Step 1.3 – Start the decision log

Create `docs/decision_log.md`. Paste this table and keep adding rows all week. These are **proposed** decisions based on the data; change any you disagree with, but write down why.

ID	Decision	Why
D01	Files live in an **external volume** on an existing ADLS external location (spec Day 1, task 4). If I lack the privilege or location, I fall back to a managed volume and record why	Spec; privileges vary by workspace
D02	Reference tables come from **Azure SQL over JDBC** if reachable (section 6.5A). Otherwise I use the two CSV extracts and label them a simulation. I never claim a live connection without evidence	Spec and README
D03	Source grain = one event row per `flight_id` per source arrival. Silver current-state grain = one row per `flight_id`	Spec Day 1 task 1
D04	Bronze keeps every value as text plus lineage columns and is never edited	Bronze is the immutable raw copy
D05	Duplicate = same `flight_id` AND identical 12 base columns. Survivor = copy from the earliest file name. Removed copies go to `silver_rejected_records` as rule R10. Nothing is deleted from Bronze	Copies are identical, so a timestamp tie-breaker cannot work
D06	`fare_amount` above 2,350 is flagged `is_fare_outlier`, not rejected	Spec: extremes are not automatically invalid
D07	Blank `flight_type` becomes `UNKNOWN` with `flight_type_was_missing = true`	Keep the row, keep the evidence
D08	`timezone` holds `USD` in 100% of rows, so it cannot be used. All timestamps are assumed to be UTC. If a valid time-zone name ever appears, it is converted	No other defensible source of truth

ID	Decision	Why
D09	The pack has no schedule data. The 'schedule revision' rule is implemented on **status revisions** from `status_updates`	Closest data available
D10	No-op status updates (previous = new) are kept in history with `is_noop = true`	Nothing is lost; easy to filter
D11	`REVERSED` is kept as a status exactly as received; its business meaning is unknown	Listed as a known limitation
D12 *(draft KPI)*	Departure delay = average `delay_minutes`. Cancellation rate = flights whose **current** status is `CANCELLED` ÷ all flights. Airport punctuality = share of flights with `delay_minutes` ⩽ 15. The 15-minute threshold is my assumption	Confirm on Day 3

Step 1.4 – Create `00_config`

In `notebooks/` create a Python notebook called `00_config` and paste this as **one cell**:

```
from pyspark.sql import functions as F, Window

CATALOG = 'YOUR_CATALOG'      # EDIT: run SHOW CATALOGS and use one you may
use (letters, digits, underscores)
SCHEMA = 'capstone03'
VOLUME = 'raw'
# EDIT for an external volume, for example
'abfss://container@account.dfs.core.windows.net/capstone03/raw'
# The folder must be inside an external location you have privileges on.
Leave as None for a managed volume.
EXTERNAL_LOCATION_URL = None

assert CATALOG != 'YOUR_CATALOG', 'Edit CATALOG at the top of this cell
first'

spark.sql(f'USE CATALOG {CATALOG}')
spark.sql(f'CREATE SCHEMA IF NOT EXISTS {SCHEMA}')
spark.sql(f'USE SCHEMA {SCHEMA}')
if EXTERNAL_LOCATION_URL:
    spark.sql(f'''CREATE EXTERNAL VOLUME IF NOT EXISTS {VOLUME} LOCATION
```

```
'{EXTERNAL_LOCATION_URL}' ''')
else:
    spark.sql(f'CREATE VOLUME IF NOT EXISTS {VOLUME}')
spark.conf.set('spark.sql.session.timeZone', 'UTC')   # every timestamp in
this project is handled as UTC

BASE = f'/Volumes/{CATALOG}/{SCHEMA}/{VOLUME}'
LANDING = f'{BASE}/landing'          # untouched originals


def t(name):
    '''Full three-part table name.'''
    return f'{CATALOG}.{SCHEMA}.{name}'


def copy_to_live(subdir, only=None):
    '''Copy CSVs from landing/<subdir> into the live folder Auto Loader
watches.
    only = list of file names to copy (None = all). Re-copying a file is
harmless.'''
    dbutils.fs.mkdirs(f'{BASE}/{subdir}')
    for f in dbutils.fs.ls(f'{LANDING}/{subdir}'):
        if f.name.endswith('.csv') and (only is None or f.name in only):
            dbutils.fs.cp(f.path, f'{BASE}/{subdir}/{f.name}')
            print('copied', f.name)


def ingest_to_bronze(subdir, table, source_label):
    '''Auto Loader: read only NEW files from <BASE>/<subdir> and append them
to a Bronze Delta table.'''
    ops = f'{BASE}/_autoloader/{table}'           # holds the schema history
and the checkpoint
    stream = (spark.readStream.format('cloudFiles')
              .option('cloudFiles.format', 'csv')
              .option('header', 'true')
              .option('cloudFiles.inferColumnTypes', 'false')      # Bronze
keeps everything as text
              .option('cloudFiles.schemaLocation', f'{ops}/schema')
              .option('cloudFiles.schemaEvolutionMode', 'addNewColumns')
              .load(f'{BASE}/{subdir}')
              .withColumn('_source_file', F.col('_metadata.file_path'))
              .withColumn('_file_modified_at',
F.col('_metadata.file_modification_time'))
              .withColumn('_ingested_at', F.current_timestamp())
              .withColumn('_source_label', F.lit(source_label)))
    query = (stream.writeStream
```

```
            .option('checkpointLocation', f'{ops}/checkpoint')
            .option('mergeSchema', 'true')                          # lets
the table accept the new column
            .trigger(availableNow=True)                             # process
what is there, then stop
            .toTable(t(table)))
    query.awaitTermination()
    return spark.table(t(table)).count()


def reset_bronze(table):
    '''DEV ONLY. Forget a Bronze table and its Auto Loader state so that
source starts from zero.'''
    spark.sql(f'DROP TABLE IF EXISTS {t(table)}')
    dbutils.fs.rm(f'{BASE}/_autoloader/{table}', True)
    print('reset', table)


print('BASE    =', BASE)
print('LANDING =', LANDING)
```

Run it. Then in a new cell run `%sql SHOW VOLUMES IN workspace.capstone03` (use your catalog name).

Expected result: it prints `BASE = /Volumes/<your_catalog>/capstone03/raw`, and `SHOW VOLUMES` lists `raw`. For an external volume, `DESCRIBE VOLUME <your_catalog>.capstone03.raw` shows the type `EXTERNAL` and your `abfss://` location. If the `CREATE EXTERNAL VOLUME` line fails, see section 12A.

Why these choices: `inferColumnTypes = false` keeps Bronze as raw text so no value is silently changed. `availableNow` is the trigger that works on serverless and makes the notebook behave like a batch job. `mergeSchema` is what lets Delta accept the new `source_schema_version` column later. The `_autoloader` folder is a sibling of the source folders, never inside them.

Step 1.5 – Upload the original files to `landing`

Unzip your pack on your computer first. In Databricks go to **Catalog → your catalog → capstone03 → raw**. Click **Create → Create directory** and make `landing`. Open `landing` and create five directories: `primary_events`, `status_updates`, `secondary_events`, `airline_id`, `airport_id`. Open each one and use **Upload to this volume** to upload the files below; this works for managed and external volumes alike. With an external volume you can instead copy the files into the matching folders of the ADLS container (Azure

Storage Explorer or the Azure portal) and they appear in the volume. One folder per source matters: Auto Loader reads a whole folder with one schema, so mixing sources would break it.

Upload this (from the unzipped pack)	Into
`datasets/adls/primary_events/` all 12 `flight_batch_NN.csv`	`landing/primary_events`
`datasets/adls/status_updates/status_updates.csv`	`landing/status_updates`
`datasets/adls/secondary_events/secondary_events.csv`	`landing/secondary_events`
`datasets/azure_sql_extracts/airline_id.csv`	`landing/airline_id`
`datasets/azure_sql_extracts/airport_id.csv`	`landing/airport_id`

Do not upload `azure_sql_ddl.sql` or `qa_manifest.json`; keep them for your documentation.

Verify in a new cell of `00_config`'s sibling notebook (or the `scratch` notebook, after `%run ./00_config`):

```
for sub in ['primary_events', 'status_updates', 'secondary_events',
'airline_id', 'airport_id']:
    files = dbutils.fs.ls(f'{LANDING}/{sub}')
    print(sub, len(files), 'files,', sum(f.size for f in files), 'bytes')
```

Expected result:

```
primary_events 12 files, 1728279 bytes
status_updates 1 files, 42748 bytes
secondary_events 1 files, 26154 bytes
airline_id 1 files, 5991 bytes
airport_id 1 files, 8118 bytes
```

If a byte count differs, the file was altered or re-saved (for example opened and saved in Excel). Delete it and upload the original again.

Step 1.6 – Commit

Commit `00_config` and `docs/decision_log.md` to `feature/day1-bronze` with a message like `day1: config notebook and decision log`. Do small commits all week; the commit history is part of your Git evidence.

5. Day 1 – Profile the raw files (notebook `01_profile_source`)

You cannot clean what you have not measured. Everything in your Day 2 rule catalogue and your decision log comes from this notebook. It also gives you the evidence for test cases **TC01** and **TC02**.

Cell 1 (this must be alone in its cell):

```
%run ./00_config
```

Cell 2 – read the 12 files one by one and stack them:

```
from functools import reduce

BASE_COLS = ['flight_id', 'event_timestamp', 'ingest_timestamp',
'airline_id', 'airport_id',
             'flight_type', 'flight_status', 'timezone', 'fare_amount',
'delay_minutes',
             'aircraft_id', 'source_system']

files = [f for f in dbutils.fs.ls(f'{LANDING}/primary_events') if
f.name.endswith('.csv')]
frames = [spark.read.option('header', 'true').csv(f.path).withColumn('_file',
F.lit(f.name)) for f in files]
raw = reduce(lambda a, b: a.unionByName(b, allowMissingColumns=True), frames)
raw.createOrReplaceTempView('raw_events')
print('raw rows:', raw.count())
```

Why file by file? If you point Spark's plain CSV reader at the whole folder, it takes the column names from one file and can silently drop or misplace the 13th column that batches 10 to 12 add. Reading each file separately makes that difference visible.

Expected result: `raw rows: 11540`

Check 1 – schema drift

```
for f in files:
    print(f.name, len(spark.read.option('header',
'true').csv(f.path).columns), 'columns')
```

Expected: `flight_batch_01` to `09` show 12 columns; `10`, `11`, `12` show 13. Write in your log: *source_schema_version added in batches 10 to 12*.

Check 2 – keys and duplicates (TC01)

```
display(spark.sql('''
    SELECT count(*) AS total_rows,
           count(DISTINCT flight_id) AS unique_flights,
           count(*) - count(DISTINCT flight_id) AS extra_rows
    FROM raw_events'''))

exact_extra = raw.count() - raw.dropDuplicates(BASE_COLS).count()
print('exact duplicate rows (all 12 columns identical):', exact_extra)

dups = (raw.groupBy('flight_id')
           .agg(F.count('*').alias('copies'),
F.countDistinct('_file').alias('files'))
           .filter('copies > 1'))
display(dups.groupBy('copies', 'files').count())
```

Expected: `11540 / 11500 / 40`; `exact duplicate rows: 40`; and the last table shows `copies = 2, files = 2` for **37** flights and `copies = 2, files = 1` for **3** flights.

What this tells you: all 40 extra rows are exact copies, none is a changed version of a flight. That is what makes the simple dedupe rule in D05 safe.

Check 3 – missing values (TC02, part 1)

```
nulls = raw.select([F.sum(F.when(F.col(c).isNull() | (F.trim(F.col(c)) ==
''), 1).otherwise(0)).alias(c)
                    for c in BASE_COLS])
display(nulls)
```

Expected: every column is `0` except `flight_type = 12`.

Check 4 – allowed values

```
for c in ['flight_type', 'flight_status', 'timezone', 'source_system']:
    display(raw.groupBy(c).count().orderBy(F.desc('count')))
```

Expected (raw, before dedupe):

Column	Values
`flight_type`	INTERNATIONAL 5,920 · DOMESTIC 5,608 · null 12
`flight_status`	FAILED 2,979 · OPEN 2,881 · COMPLETED 2,856 · PENDING 2,824

Column	Values
`timezone`	USD 11,540
`source_system`	ADLS_EVENT 11,540

Two things to write down: there is **no CANCELLED** status in the flight events, and `timezone` is a currency code in every row.

Check 5 – extreme values (TC02, part 2)

```
num = (raw.withColumn('fare', F.col('fare_amount').cast('double'))
          .withColumn('delay', F.col('delay_minutes').cast('double')))

display(num.select(F.min('fare').alias('fare_min'), F.expr('percentile(fare,
0.5)').alias('fare_median'),
                   F.expr('percentile(fare, 0.99)').alias('fare_p99'),
F.max('fare').alias('fare_max'),
                   F.min('delay').alias('delay_min'),
F.avg('delay').alias('delay_avg'), F.max('delay').alias('delay_max')))

q1, q3 = num.approxQuantile('fare', [0.25, 0.75], 0.001)
print('Q1', q1, 'Q3', q3, 'upper fence (Q3 + 3 x IQR)', q3 + 3 * (q3 - q1))

display(num.filter('fare > 2350').orderBy(F.desc('fare')).select('flight_id',
'fare', 'delay', '_file'))
print('rows above 2,350:', num.filter('fare > 2350').count())
```

Expected: fare min 100.16, median about 601.4, p99 about 1,092.9, max 77,083.50. Delay min 0, average about 44.99, max 90. Q1 about 348.5, Q3 about 851.2, fence about 2,359. **Rows above 2,350: 35.**

The fence says about 2,359 and the decision log says 2,350. Both flag the same 35 rows, because nothing sits between 1,099.99 (largest normal fare) and 8,110.50 (smallest extreme one). I use a fixed 2,350 on purpose: a threshold that is recalculated on every run would flag different rows as data grows, and your reruns would stop being repeatable.

Check 6 – timestamps

```
display(spark.sql('''
    SELECT min(event_timestamp) AS first_event, max(event_timestamp) AS
last_event,
           count_if(length(event_timestamp) != 19) AS wrong_length
    FROM raw_events'''))
```

Expected: `2026-07-01T00:02:00`, `2026-08-01T22:36:00`, `0`.

Check 7 – the other four sources and key coverage

```
cdc = spark.read.option('header', 'true').csv(f'{LANDING}/status_updates/')
sec = spark.read.option('header', 'true').csv(f'{LANDING}/secondary_events/')
airline = spark.read.option('header', 'true').csv(f'{LANDING}/airline_id/')
airport = spark.read.option('header', 'true').csv(f'{LANDING}/airport_id/')
flights = raw.select('flight_id').distinct()

display(cdc.groupBy('new_status').count())
print('CDC rows:', cdc.count(), '| distinct flights:',
cdc.select('flight_id').distinct().count())
print('CDC time range:', cdc.agg(F.min('change_timestamp'),
F.max('change_timestamp')).first())
print('CDC no-op rows (previous = new):', cdc.filter('previous_status =
new_status').count())
print('CDC flights not in events:', cdc.join(flights, 'flight_id',
'left_anti').count())
print('events with unknown airline:', raw.join(airline, 'airline_id',
'left_anti').count())
print('events with unknown airport:', raw.join(airport, 'airport_id',
'left_anti').count())
print('secondary rows:', sec.count(), '| not linked to a flight:',
sec.join(flights, 'flight_id', 'left_anti').count())
print('airlines:', airline.count(), '| airports:', airport.count())
```

Expected: COMPLETED 176, REVERSED 173, CANCELLED 171; `CDC rows: 520 | distinct flights: 520`; time range `2026-08-10T00:00:00` to `2026-08-10T08:39:00`; no-ops 45; every 'not in' or 'unknown' count is 0; `secondary rows: 310`; airlines 110, airports 150.

Write up the profile

Create `docs/source_profile.md` with the numbers from this section in a small table (check, expected, actual, pass or fail). Also paste the 6 'Corrections' from section 0 as **findings**. Compare your numbers with `qa_manifest.json` (40 duplicates, 12 nulls, 35 outliers, 520 CDC rows, 310 secondary rows, 260 reference rows). That file is your answer key.

Evidence this produces: TC01 (11,540 rows, 40 duplicates, 11,500 unique) and TC02 (12 optional nulls, 35 extremes documented).

6. Day 1 – Load Bronze (notebook `02_bronze_ingest`)

6.1 What Auto Loader does, in plain words

Auto Loader watches a folder. The first time it runs it loads every file there; after that it loads only files it has not seen. It remembers which files are done in a **checkpoint** (a folder it manages), and it remembers the column layout in the **schema location**. This is what makes the pipeline *restart-safe*: if a run dies halfway, the next run carries on without losing or repeating files.

Cell 1: `%run ./00_config` (alone in its cell).

6.2 Round 1 – batches 01 to 09 arrive

```
batches_1 = [f'flight_batch_{i:02d}.csv' for i in range(1, 10)]
copy_to_live('primary_events', only=batches_1)
n = ingest_to_bronze('primary_events', 'bronze_flight_events',
'ADLS_EVENT_FILES')
print('bronze_flight_events rows:', n)
print(spark.table(t('bronze_flight_events')).columns)
```

The first run can take a couple of minutes while serverless compute starts.

Expected result: nine `copied ...` lines, then `bronze_flight_events rows: 9000`. The column list ends with `source_system, _rescued_data, _source_file, _file_modified_at, _ingested_at, _source_label` and does **not** contain `source_schema_version` yet.

6.3 Round 2 – batches 10 to 12 arrive with a new column

```
batches_2 = [f'flight_batch_{i:02d}.csv' for i in range(10, 13)]
copy_to_live('primary_events', only=batches_2)
n = ingest_to_bronze('primary_events', 'bronze_flight_events',
'ADLS_EVENT_FILES')
print('bronze_flight_events rows:', n)
```

Expected result: this run most likely **stops with an error** that mentions a new field or column, `source_schema_version`. That is not a bug. Auto Loader has detected schema drift, saved the new schema, and stopped on purpose so that a human or a job retry decides what happens next. **Take a screenshot of the error and note the time in your decision log.** This is your schema-drift evidence.

Now run the *same cell again*.

Expected result: `bronze_flight_events rows: 11540`. If the first run succeeded without an error, that is fine too; record that in your log and carry on.

Verify the new column:

```
bronze = spark.table(t('bronze_flight_events'))
print('source_schema_version' in bronze.columns)
display(bronze.groupBy('source_schema_version').count())
```

Expected: `True`; `v2` = 2,540 rows and `null` = 9,000 rows (the first nine batches never had the column).

6.4 Replay safety

Run the Round 2 ingest cell a third time without copying any new file.

Expected: it finishes quickly and still returns `11540`. Re-running added nothing and duplicated nothing, because the checkpoint knows every file is done. Note this as evidence for TC03 and TC06 (the full tests come on Day 4).

Optional but useful: run `DESCRIBE HISTORY workspace.capstone03.bronze_flight_events` (use your catalog). You will see one version per micro-batch with the operation `STREAMING UPDATE`. Screenshot it.

6.5 The other four sources

```
for sub in ['status_updates', 'secondary_events', 'airline_id',
'airport_id']:
    copy_to_live(sub)

print(ingest_to_bronze('status_updates', 'bronze_status_cdc',
'ADLS_CDC_FILES'))
print(ingest_to_bronze('secondary_events', 'bronze_secondary_events',
'ADLS_AUX_FILES'))
print(ingest_to_bronze('airline_id', 'bronze_airline_id',
'CSV_EXTRACT_SIMULATING_AZURE_SQL'))
print(ingest_to_bronze('airport_id', 'bronze_airport_id',
'CSV_EXTRACT_SIMULATING_AZURE_SQL'))
```

Expected: `520`, `310`, `110`, `150`.

The `_source_label` column of the two lookup tables says `CSV_EXTRACT_SIMULATING_AZURE_SQL`. That is how you keep the promise in D02: you never claim a live Azure SQL connection.

6.5A Optional: load the lookups from a real Azure SQL Database

Do this **instead of** the two lookup lines in 6.5 (`airline_id` and `airport_id`), only if you can reach an Azure SQL Database from your compute. Allow about an hour. I could not test this against a database, so treat the first run as a test and read the error text. Never write 'live Azure SQL' in your documents unless you actually ran this and saved the evidence.

1. **Database.** Use the one you were given, or create a small Azure SQL Database. Run `azure_sql_ddl.sql` from the pack (it creates `dbo.airline_id` and `dbo.airport_id`), then import `airline_id.csv` and `airport_id.csv` with the Azure Data Studio import wizard or the portal query editor.
2. **Network.** The usual blocker. Your Databricks compute must be allowed through the Azure SQL firewall (an IP allow rule for classic clusters, or the workspace's network configuration for serverless). If the connection times out, ask your admin.
3. **Secrets.** Never paste a password into a notebook. Ask your admin for, or create, a secret scope named `capstone03` holding `sql-host`, `sql-db`, `sql-user`, `sql-password` (Databricks CLI: `databricks secrets create-scope capstone03`, then `databricks secrets put-secret capstone03 sql-password`; the exact command spelling depends on your CLI version).
4. **Load.**

```
host = dbutils.secrets.get('capstone03', 'sql-host')       # for example
myserver.database.windows.net
db = dbutils.secrets.get('capstone03', 'sql-db')
user = dbutils.secrets.get('capstone03', 'sql-user')
pwd = dbutils.secrets.get('capstone03', 'sql-password')
jdbc_url = f'jdbc:sqlserver://{host}:1433;database=
{db};encrypt=true;trustServerCertificate=false;loginTimeout=30'


def load_lookup_from_sql(source_table, bronze_table):
    '''Snapshot a SQL table into Bronze: all columns as text, plus the same
audit columns.'''
    df = (spark.read.format('jdbc')
          .option('url', jdbc_url).option('dbtable', source_table)
          .option('user', user).option('password', pwd).load())
    df = df.select([F.col(c).cast('string').alias(c) for c in df.columns])
    df = (df.withColumn('_rescued_data', F.lit(None).cast('string'))
            .withColumn('_source_file', F.lit(f'azure_sql:{source_table}'))
            .withColumn('_file_modified_at', F.lit(None).cast('timestamp'))
            .withColumn('_ingested_at', F.current_timestamp())
            .withColumn('_source_label', F.lit('AZURE_SQL_JDBC')))
    df.write.mode('overwrite').option('overwriteSchema',
```

```
'true').saveAsTable(t(bronze_table))
    return spark.table(t(bronze_table)).count()


print(load_lookup_from_sql('dbo.airline_id', 'bronze_airline_id'))
print(load_lookup_from_sql('dbo.airport_id', 'bronze_airport_id'))
```

Expected: `110` and `150`, and `_source_label` is `AZURE_SQL_JDBC`. Evidence to save: a screenshot of `SELECT count(*)` in Azure SQL beside this output, with the secret values hidden.

Two consequences. These two lookup tables are *snapshots replaced on each load*, so **leave them out of the append-only loop in step 6.7** (append-only blocks overwrite). Their history is still available through Delta time travel. Also update D02 to say 'live JDBC', and drop the CSV-simulation wording from `docs/limitations.md`.

6.6 Bronze audit (your Day 1 evidence)

```
display(spark.sql(f'''
    SELECT 'bronze_flight_events' AS table_name, count(*) AS row_count FROM
{t('bronze_flight_events')} UNION ALL
    SELECT 'bronze_status_cdc', count(*) FROM {t('bronze_status_cdc')} UNION
ALL
    SELECT 'bronze_secondary_events', count(*) FROM
{t('bronze_secondary_events')} UNION ALL
    SELECT 'bronze_airline_id', count(*) FROM {t('bronze_airline_id')} UNION
ALL
    SELECT 'bronze_airport_id', count(*) FROM {t('bronze_airport_id')}'''))

bronze = spark.table(t('bronze_flight_events'))
display(bronze.withColumn('file', F.element_at(F.split('_source_file', '/'),
-1))
              .groupBy('file').count().orderBy('file'))

print('rows with rescued data:', bronze.filter('_rescued_data IS NOT
NULL').count())
display(bronze.filter(F.col('flight_id') == 'FLIGHT-
0000501').select('flight_id', '_source_file', '_ingested_at'))
```

Expected:

- Row counts 11,540 / 520 / 310 / 110 / 150, which add up to **12,630**, the `rows_total` in the manifest.
- Eleven files with 1,000 rows and `flight_batch_12.csv` with 540.

- `rows with rescued data: 0`. If it is not 0, a row had more or fewer fields than the header; look at those rows before continuing.
- `FLIGHT-0000501` appears **twice**, once from `flight_batch_01.csv` and once from `flight_batch_04.csv`. Bronze keeps both on purpose.

6.7 Make Bronze append-only and prove it

```
for name in ['bronze_flight_events', 'bronze_status_cdc',
'bronze_secondary_events',
             'bronze_airline_id', 'bronze_airport_id']:
    spark.sql(f'''ALTER TABLE {t(name)} SET TBLPROPERTIES ('delta.appendOnly'
= 'true')''')

try:
    spark.sql(f'''UPDATE {t('bronze_flight_events')} SET flight_type = 'X'
WHERE flight_id = 'FLIGHT-0000501' ''')
except Exception as e:
    print('Blocked as expected:', str(e)[:200])
```

Expected: a message saying the table only allows appends. Screenshot it: that is proof that Bronze is immutable.

If you ever need to start a Bronze source again from zero during development, first run `ALTER TABLE ... SET TBLPROPERTIES ('delta.appendOnly' = 'false')` on that table, then `reset_bronze('<table name>')`, then re-run the ingest.

6.8 Day 1 documents (keep them short)

Put these in your repo `README.md` and `docs/`:

1. Architecture one-pager – paste the diagram from section 2 and add one line per box.

2. Source-to-target mapping (Bronze):

Source	Live folder	Bronze table	Business key	Added columns
Flight events (12 CSV)	`primary_events`	`bronze_flight_events`	`flight_id` (not unique in Bronze)	`_rescued_data`, `_source_file`, `_file_modified_at`, `_ingested_at`, `_source_label`
Status updates	`status_updates`	`bronze_status_cdc`	`flight_id` + `source_sequence`	same
Secondary events	`secondary_events`	`bronze_secondary_events`	`event_id`	same
Airline lookup	`airline_id`	`bronze_airline_id`	`airline_id`	same
Airport lookup	`airport_id`	`bronze_airport_id`	`airport_id`	same

3. **`docs/limitations.md`** – start it now: any fallback you had to take, with the reason (managed volume instead of an external volume, CSV simulation instead of live Azure SQL); `timezone` unusable; no schedule data in the pack; `REVERSED` has no documented meaning; workspace limits you discover (add Genie and job-alert availability when you reach Day 4 and 5).

6.9 Commit and Day 1 evidence checklist

Commit your notebooks and docs to `feature/day1-bronze`, open a pull request to `main` and merge it yourself (a self-merged PR is allowed for this solo project). Then check that you have all of these:

- [ ] `docs/decision_log.md` with D01 to D12
- [ ] `docs/source_profile.md` with expected vs actual for TC01 and TC02
- [ ] Screenshot of the schema-drift error and of the successful rerun
- [ ] Bronze audit output (12,630 rows total) and per-file counts
- [ ] Screenshot of the append-only block
- [ ] Repo with notebooks `00_config`, `01_profile_source`, `02_bronze_ingest`
- [ ] README with architecture and mapping; `docs/limitations.md` started
- [ ] At least three commits and one merged pull request

6A. Day 1 – Design documents (BRS, HLD, LLD, ERD, data dictionary, mapping)

The spec makes these part of Day 1, and requirements plus architecture carry 10 of the 100 points. Working solo, you are both the analyst and the architect. Keep each document to one or two pages in `docs/`. They cost roughly 2 extra hours, so Day 1 becomes a 9 to 10 hour day. Write the **BRS and HLD early** (they need only the spec and the profile), and the **LLD, ERD, data dictionary and mapping after Bronze is loaded**, when the real column names are in front of you. Do not skip them to save time; they are cheaper than the marks.

6A.1 Business requirements (BRS) – `docs/01_BRS.md`

The client's specification is the BRS. Your job is to restate it in your own words, add IDs so you can trace every requirement to a test later, and list what is unclear. There is no live workshop, so the open questions below are what you would have asked the client.

Business problem and objective. Flight events, status revisions and business keys arrive from several systems and are handled inconsistently. Operations cannot explain exceptions, trace a record to its source, or reconcile totals. Objective: improve flight punctuality and support delay attribution through a governed lakehouse.

Stakeholders

Stakeholder	What they need from the platform
Operations manager	Trusted delay, cancellation and punctuality numbers by day and airport
Analytics lead	Clean Gold tables and a dashboard to build on
Data governance owner	Traceability, documented exceptions, access control
Platform operations	Restart-safe, monitored, re-runnable jobs and a runbook
Client product owner	Acceptance evidence and a clear sign-off request

Requirements (use these IDs in your traceability matrix on Day 5)

ID	Requirement	Planned evidence
BR-F01	Ingest flight event files incrementally, including a controlled schema change	Bronze load, schema-drift screenshot
BR-F02	Ingest airline and airport reference data (Azure SQL; simulated here)	Lookup Bronze tables, TC07
BR-F03	Process status changes (CDC) with history	Section 10, TC04
BR-F04	Publish validated Gold marts: departure delay, cancellation rate, airport punctuality	Day 3, TC08, TC09
BR-F05	Keep immutable raw history with source metadata	Append-only proof (6.7)
BR-F06	Apply auditable data-quality rules with quarantine	Rule catalogue, rejected table, TC01, TC02, TC05
BR-F07	Match flight events to schedule revisions by effective time; convert local time to UTC	Section 10.3, tests U01, U10
BR-N01	Restart-safe pipelines	TC03, TC06

ID	Requirement	Planned evidence
BR-N02	Least-privilege access	Day 4 grants
BR-N03	Traceable source metadata	`_source_file`, `_ingested_at`
BR-N04	Freshness reporting	Day 4 monitoring
BR-N05	Reproducible jobs	Git, Workflows (Day 4)
BR-N06	Failure alerts	Day 4 (depends on your workspace settings)

Acceptance conditions from the spec: expected-versus-actual reconciliation; no duplicate business keys in curated current-state tables; documented exceptions; a repeatable end-to-end workflow; a client walkthrough.

Out of scope (from the spec): live transaction decisions, production model serving, enterprise CI/CD, real personal data, external notification integrations.

KPI definitions (draft, confirm on Day 3)

KPI	Business definition	Formula	Grain
Departure delay	How late flights leave, on average	average of `delay_minutes`	day and airport
Cancellation rate	Share of flights that end up cancelled	flights with current status `CANCELLED` ÷ all flights	day and airport
Airport punctuality	Share of flights leaving within the threshold	flights with `delay_minutes` ⩽ 15 ÷ all flights	day and airport

Open questions for the client, with my working assumption (write your own assumption beside each)

1. What does the status `REVERSED` mean? *Assumption: keep as received, do not count it as cancelled.*
2. Which time zone are the timestamps in? *Assumption: UTC.*
3. Is 15 minutes the right punctuality threshold? *Assumption: yes, a common on-time convention; this one is mine.*
4. Does `FAILED` count as a cancellation? *Assumption: no.*

5. Should flights still OPEN or PENDING count in the cancellation-rate denominator? *Assumption: yes, all flights.*
6. Which currency is `fare_amount` in? *Assumption: unknown; not used in the three KPIs.*
7. Where do schedule revisions come from? *Assumption: not provided; status revisions are used instead.*

RAID starter (risks, assumptions, issues, dependencies)

Type	Item	Impact	Mitigation
Risk	Compute quota, cluster policy or auto-termination interrupts work	Lost working time	Start early; keep every step re-runnable
Risk	Some features (Genie, alerts, grants) may be disabled by your admin	Lost marks	Document the constraint; the spec allows it
Issue	`timezone` holds `USD`	UTC conversion impossible	Assume UTC, flag every row, list as limitation
Issue	No schedule data in the pack	Rule BR-F07 only partly demonstrable	Apply it to status revisions, say so
Issue	`REVERSED` has no stated meaning	KPI ambiguity	Open question 1
Assumption	Punctuality threshold 15 minutes	KPI value depends on it	Parameterise on Day 3
Dependency	GitHub access and token	No Git evidence	Set up in step 1.2

6A.2 High-level design (HLD) – `docs/02_HLD.md`

One page: the picture, then what each box is, then how it differs from the spec's ideal architecture.

```
flowchart LR
  A[CSV files in landing folder] --> B[Live source folders]
  B --> C[Auto Loader]
  C --> D[Bronze Delta tables]
  D --> E[Silver Delta tables]
  E --> F[Gold Delta tables]
  F --> G[SQL warehouse and AI BI dashboard]
  H[Workflows - Day 4] -.-> C
  H -.-> E
  H -.-> F
  I[GitHub repository] -.-> H
```

Spec component	What you use	Note
ADLS Gen2 landing	ADLS container behind an existing external location, exposed as a Unity Catalog **external volume** (managed volume as fallback)	Needs `CREATE EXTERNAL VOLUME`, `READ FILES`, `WRITE FILES`
Auto Loader	Auto Loader, `availableNow` trigger	Directory listing mode is fine at this file count
Azure SQL Database	JDBC read with credentials from a secret scope (CSV extracts as fallback)	Label which one you used; evidence required for 'live'
Unity Catalog	One schema `capstone03` in your catalog	Grants are configured on Day 4
Bronze, Silver, Gold Delta	Same	Same
Databricks Workflows	Jobs with retries and failure notifications (Day 4)	Job compute or serverless, per your workspace
SQL warehouse and AI/BI dashboards	A SQL warehouse of the size your admin allows	A small size is enough for this data
Genie and MLflow	Day 5	Confirm Genie is enabled now (step 1.1)
Git folders	GitHub repository plus a Databricks Git folder	Follow your organisation's pull-request rules

How data moves. Files land untouched in `landing`, are copied to a live folder, Auto Loader appends them to Bronze with file name and load time, Silver cleans, deduplicates and applies status changes, Gold publishes the three KPIs, and the dashboard reads Gold only.

Cross-cutting design. Restart safety comes from Auto Loader checkpoints and re-runnable writes. Traceability comes from the audit columns and the rejected-records table. Security comes from Unity Catalog grants (Day 4). Observability comes from Delta table history and row-count reconciliations.

6A.3 Low-level design (LLD) – `docs/03_LLD.md`

The HLD says *what* the parts are; the LLD says *exactly how each table is built*. Paste these tables and fill in anything you change.

Naming conventions

Thing	Convention
Catalog and schema	`workspace.capstone03` (one schema for all layers, as the implementation guide says)
Tables	`<layer>_<entity>`, snake_case: `bronze_`, `silver_`, `fact_`, `dim_`, `mart_`
Columns	snake_case. Bronze keeps source names unchanged. Audit columns start with an underscore (`_source_file`). Time columns that are UTC end in `_ts_utc`
Boolean flags	`is_...` or `..._was_missing`
Folders	`raw/landing/<source>` originals, `raw/<source>` live, `raw/_autoloader/<table>/schema` and `/checkpoint`
Notebooks	`NN_purpose`, run in number order
Git	branches `feature/dayN-topic`; commit messages `dayN: what changed`

Table specifications

Table	Layer	Grain (one row per…)	Key	Load pattern	Built in
`bronze_flight_events`	Bronze	source arrival of a flight event	none enforced (`flight_id` repeats)	Auto Loader append, `availableNow`	`02`
`bronze_status_cdc`	Bronze	status change as received	`flight_id` + `source_sequence`	same	`02`
`bronze_secondary_events`	Bronze	auxiliary event	`event_id`	same	`02`
`bronze_airline_id` / `bronze_airport_id`	Bronze	lookup row as received	`airline_id` / `airport_id`	same	`02`
`silver_flight_events`	Silver	distinct flight (deduplicated)	`flight_id`	full rebuild from Bronze (overwrite)	`04`
`silver_flight_current`	Silver	distinct flight with latest status	`flight_id`	insert-only seed + guarded MERGE	`05`
`silver_status_history`	Silver	status state of a flight over time	`flight_id` + `source_sequence`	insert-only MERGE	`05`
`silver_rejected_records`	Silver	rejected or removed record	none (`stage`, `rule_id`, `flight_id`)	delete by stage, then append	`04`, `05`
`silver_flight_event_status_asof`	Silver	flight event	`flight_id`	overwrite	`05`

Gold plan (draft, confirm on Day 3)

Table	Grain	Key	Main columns
`dim_airline`	airline	`airline_id`	name, region, active flag, risk tier
`dim_airport`	airport	`airport_id`	name, region, active flag, risk tier
`fact_flight_activity`	flight	`flight_id`	airline, airport, event date, delay, fare, current status, outlier flag
`fact_status_transitions`	status change	`flight_id` + `source_sequence`	from status, to status, effective time, reason
`mart_flight_daily`	day and airport	`event_date` + `airport_id`	flights, average delay, cancellation rate, punctuality %

Draft decision **D17**: use the natural keys `airline_id` and `airport_id` instead of surrogate keys, because they are unique, populated in 100% of rows and never change in this data. Write that down as a deliberate choice on Day 3.

Reliability design

- *Restart safety:* Auto Loader checkpoints; every Silver write is an overwrite, an insert-only MERGE, or a guarded MERGE, so any step can be run twice.
- *Failure handling:* invalid records are quarantined with a reason, never silently dropped; a failed job is fixed and re-run.
- *Observability:* Delta table history (`DESCRIBE HISTORY`), row-count reconciliation at every layer, rejected counts per rule.
- *Security:* Unity Catalog grants on the Gold tables (Day 4); no tokens or secrets in notebooks.

6A.4 Entity-relationship diagram (ERD) – `docs/04_ERD.md`

Paste this into your `README.md` or `docs/04_ERD.md` inside a code block marked `mermaid`. GitHub draws it automatically. It shows Bronze and Silver now plus the two Gold dimensions; add the Gold facts on Day 3.

```
erDiagram
  DIM_AIRLINE ||--o{ SILVER_FLIGHT_CURRENT : operates
  DIM_AIRPORT ||--o{ SILVER_FLIGHT_CURRENT : hosts
  SILVER_FLIGHT_EVENTS ||--|| SILVER_FLIGHT_CURRENT : seeds
  SILVER_FLIGHT_CURRENT ||--o{ SILVER_STATUS_HISTORY : has
  SILVER_FLIGHT_CURRENT ||--o{ BRONZE_SECONDARY_EVENTS : linked_by_flight_id
  SILVER_FLIGHT_EVENTS {
    string flight_id PK
    timestamp event_ts_utc
    string airline_id FK
    string airport_id FK
    string flight_type
    string flight_status
    decimal fare_amount
    int delay_minutes
    boolean is_fare_outlier
  }
  SILVER_FLIGHT_CURRENT {
    string flight_id PK
    string airline_id FK
    string airport_id FK
    string current_status
    timestamp status_updated_at
    bigint last_source_sequence
  }
  SILVER_STATUS_HISTORY {
    string flight_id FK
    bigint source_sequence
```

```
    timestamp effective_ts_utc
    string new_status
    string record_type
  }
  SILVER_REJECTED_RECORDS {
    string stage
    string rule_id
    string flight_id
    string source_file
  }
  BRONZE_SECONDARY_EVENTS {
    string event_id PK
    string flight_id FK
    string event_type
  }
  DIM_AIRLINE {
    string airline_id PK
    string region
    string active_flag
  }
  DIM_AIRPORT {
    string airport_id PK
    string region
    string active_flag
  }
```

Also write three sentences under it: one flight has many status-history rows; one airline or airport has many flights; the rejected table points back to a flight and a source file but is not a foreign-key child, because rejected rows may have a bad or missing `flight_id`.

6A.5 Data dictionary – `docs/05_data_dictionary.md`

Built from your real files. Columns marked *(my reading)* are interpretations the source does not state; keep that label so you never present a guess as fact.

Flight events (`primary_events` → `bronze_flight_events` → `silver_flight_events`)

Source column	Silver column and type	Meaning	Observed values	Nullable	Example
`flight_id`	`flight_id` string	Business key of the flight	`FLIGHT-` + 7 digits; 11,500 distinct	No	FLIGHT-0005877
`event_timestamp`	`event_ts_utc` timestamp	When the flight event happened. Text with no zone	2026-07-01 to 2026-08-01	No	2026-07-17T07:46:00

Source column	Silver column and type	Meaning	Observed values	Nullable	Example
`ingest_timestamp`	`ingest_ts_utc` timestamp	When the source recorded the event *(my reading)*	1 to 25 minutes after the event, average 13	No	2026-07-17T07:47:00
`airline_id`	`airline_id` string	Airline key, joins to the airline lookup	110 distinct	No	AIRLINE_ID-0047
`airport_id`	`airport_id` string	Airport key, joins to the airport lookup	150 distinct	No	AIRPORT_ID-0027
`flight_type`	`flight_type` string	Domestic or international	DOMESTIC, INTERNATIONAL; blank in 12 rows (filled `UNKNOWN`)	Yes	DOMESTIC
`flight_status`	`flight_status` string	Status reported on this event	OPEN, PENDING, COMPLETED, FAILED	No	PENDING
`timezone`	`timezone_raw`, `tz_used`, `timezone_is_valid`	Intended zone of the timestamps. Unusable: holds `USD`	USD in every row	No	USD
`fare_amount`	`fare_amount` decimal(12,2)	Fare amount; currency is not stated	100.16 to 1,099.99 normally; 35 extremes up to 77,083.50	No	800.0
`delay_minutes`	`delay_minutes` int	Departure delay in minutes *(my reading)*	0 to 90, average about 45	No	65
`aircraft_id`	`aircraft_id` string	Aircraft identifier	65 distinct	No	AIRCRAFT_ID-0027
`source_system`	`source_system` string	Origin system label	ADLS_EVENT	No	ADLS_EVENT
`source_schema_version`	same	Added in batches 10 to 12	`v2`; empty for batches 1 to 9	Yes	v2

Status updates (`bronze_status_cdc`)

Column	Meaning	Observed values
`flight_id`	Flight the change applies to	520 distinct, all exist in the events
`change_timestamp`	When the new status takes effect	2026-08-10 00:00 to 08:39
`previous_status`	Status before the change	OPEN, PENDING, COMPLETED, FAILED; matches the event status in 100% of rows

Column	Meaning	Observed values
`new_status`	Status after the change	COMPLETED 176, REVERSED 173, CANCELLED 171
`change_reason`	Why it changed	OPERATIONS_UPDATE 180, CUSTOMER_ACTION 172, SYSTEM_REVIEW 168
`source_sequence`	Ordering number from the source	1 to 520, unique
`source_system`	Origin label	ADLS_CDC

Secondary events (`bronze_secondary_events`)

Column	Meaning	Observed values
`event_id`	Event key	`SEC-` + 6 digits, 310 unique
`flight_id`	Flight it belongs to	310 distinct, all exist; `event_timestamp` equals that flight's event time
`event_timestamp`	Event time	2026-07-01 to 2026-07-27
`related_entity_id`	Related entity *(my reading)*	Always the flight's own airline_id
`event_type`	Kind of event	CHECK 117, ALERT 97, REVIEW 96
`event_value`	Numeric value, meaning not stated	0.25 to 99.62
`source_system`	Origin label	ADLS_AUX

Lookups (`bronze_airline_id` , `bronze_airport_id`) have the same six columns: the key (`airline_id` or `airport_id`), `display_name` (for example 'Airline 0001'), `region` (NORTH, SOUTH, EAST, WEST), `active_flag` (Y or N), `effective_date` (2026-01-01 for every row) and `risk_tier` (LOW, MEDIUM, HIGH).

Audit columns added by you (every Bronze table): `_rescued_data` (Auto Loader puts unparseable content here; should be empty), `_source_file` (full path of the file), `_file_modified_at` , `_ingested_at` (when you loaded it), `_source_label` (which source system label applies, including the simulation label).

6A.6 Source-to-target mapping for Silver – `docs/06_source_to_target.md`

Silver column	From Bronze column	Transformation	Rule
`flight_id`	`flight_id`	trim; blank becomes NULL	R01, R10, R11
`event_ts_utc`	`event_timestamp`	must match the format, then cast to timestamp, then convert from `tz_used` to UTC	R02, D08
`ingest_ts_utc`	`ingest_timestamp`	same	R03
`airline_id` , `airport_id`	same names	trim	R07
`flight_type`	`flight_type`	blank becomes `UNKNOWN`	F03
`flight_type_was_missing`	`flight_type`	true when it was blank	F03
`flight_status`	`flight_status`	trim; must be one of four values	R08
`timezone_raw`	`timezone`	copied unchanged	F04
`tz_used` , `timezone_is_valid`	`timezone`	real zone name or else UTC; flag	F04, D08
`fare_amount`	`fare_amount`	cast to decimal(12,2)	R06
`delay_minutes`	`delay_minutes`	cast to integer; negative is rejected	R04, R05
`is_fare_outlier`	`fare_amount`	true when above 2,350	F01
`is_delay_outlier`	`delay_minutes`	true when above 180	F02
`aircraft_id` , `source_system`	same names	trim	none
`source_schema_version`	same	NULL when the column does not exist	none
`_row_hash`	the 12 base columns	SHA-256 fingerprint	R10
`_source_file` , `_ingested_at`	same names	carried through for lineage	none

Silver column	From Bronze column	Transformation	Rule
`_processed_at`	none	time Silver was built	none

Status updates to Silver:

Target	Source	Logic
`silver_status_history` (`record_type = INITIAL`)	`silver_flight_events`	one row per flight: its reported status at event time, `source_sequence = 0`
`silver_status_history` (`record_type = CDC`)	`bronze_status_cdc`	every valid change; `is_noop` true when previous = new
`silver_flight_current.current_status`	latest valid change per flight	applied only if newer than `status_updated_at`
`silver_rejected_records` (`stage = CDC`)	invalid `bronze_status_cdc` rows	rules C01 to C05

Lookups to Gold (planned, Day 3): `bronze_airline_id` → `dim_airline`, `bronze_airport_id` → `dim_airport`, with the same column names and `active_flag`, `risk_tier` carried across.

6A.7 Bronze source-contract tests

The spec lists *source contract tests* (required columns, types, timestamps, file arrival, allowed schema changes). Add this as the **last cell of `02_bronze_ingest`**. I ran this exact logic against your 12 files locally.

```
bronze = spark.table(t('bronze_flight_events'))
required = ['flight_id', 'event_timestamp', 'ingest_timestamp', 'airline_id',
'airport_id', 'flight_type',
            'flight_status', 'timezone', 'fare_amount', 'delay_minutes',
'aircraft_id', 'source_system']
audit = ['_rescued_data', '_source_file', '_file_modified_at',
'_ingested_at', '_source_label']
ts_regex = '^[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}$'
extra = set(bronze.columns) - set(required) - set(audit)

tests = [
    ('required columns present', all(c in bronze.columns for c in required)),
    ('only allowed extra column is source_schema_version', extra <=
{'source_schema_version'}),
    ('no row carries rescued data', bronze.filter('_rescued_data IS NOT
NULL').count() == 0),
    ('every row has a source file and load time', bronze.filter('_source_file
IS NULL OR _ingested_at IS NULL').count() == 0),
    ('all timestamps match the contract format',
     bronze.filter(~F.col('event_timestamp').rlike(ts_regex) |
~F.col('ingest_timestamp').rlike(ts_regex)).count() == 0),
    ('all 12 batch files arrived',
bronze.select('_source_file').distinct().count() == 12),
]
for name, ok in tests:
    print('PASS' if ok else 'FAIL', '-', name)
assert all(ok for _, ok in tests)
```

Expected: six `PASS` lines. Save the output in `docs/evidence/day1/`.

6A.8 Lineage evidence

The spec asks you to demonstrate the first micro-batch and lineage. Open **Catalog → workspace → capstone03 →** **`bronze_flight_events`** and look at the **Lineage** tab. Screenshot whatever it shows, even if it is sparse for file-based sources. After Day 2, open `silver_flight_events` and screenshot its lineage, which should show the Bronze table feeding it. The lineage you fully control is row-level: `_source_file` and `_ingested_at` on every Bronze row, and `source_file` plus the original row as JSON in `silver_rejected_records`. Say that in the walkthrough.

6A.9 Spec cross-check for Days 1 and 2

I went through the spec and the implementation guide line by line. This is where each requirement is covered and where you deviate.

Requirement (spec or guide)	Where covered	Note
Read BRS, define source grain (D1)	6A.1, D03	No live workshop; you act as analyst
HLD, LLD, ERD, source-to-target mapping (D1)	6A.2, 6A.3, 6A.4, 6.8, 6A.6	Gold parts finished on Day 3
Data dictionary (D1)	6A.5	
Profile all files: counts, types, 12 nulls, 40 duplicates, 35 extremes, status, key coverage (D1)	Section 5	TC01, TC02
Git folders, branch naming, PR checklist (D1)	1.2, 3A, 6.9	
One Unity Catalog schema; external volume (D1)	1.4	**Deviation** only if you fall back to a managed volume (D01)
Auto Loader with schema and checkpoint location; file name and load time (D1)	1.4, 6.2 to 6.4	
Azure SQL reference tables or labelled simulation (D1)	6.5	**Deviation** only if you fall back to the CSV simulation (D02)
Bronze audit, first micro-batch, lineage (D1)	6.6, 6A.8	
Source contract tests (spec section 7)	6A.7	
Silver schema and DQ rule catalogue with ID, severity, action (D2)	Section 7	
Parse timestamps and numbers; preserve raw values; quarantine invalid mandatory fields (D2)	Sections 8, 9	Raw lives in Bronze; rejects also keep the raw row as JSON
Deterministic dedupe, duplicate audit counts (D2)	Sections 7, 9.1	
CDC in sequence and time order; late events never overwrite newer status (D2)	Section 10, tests U08, U11, U12	Late cases are synthetic, the data has none

Requirement (spec or guide)	Where covered	Note
Domain rule: schedule revisions by effective time, local time to UTC (D2)	Section 10.3	**Deviation:** no schedule data; no usable time zone
Unit tests: valid, invalid, duplicate, late, out of order (D2)	Section 11	
Evidence: Silver schema, rejected rows, status history, tests, before/after counts (D2)	9.1, 10, 11, 13	

Not covered here on purpose: Gold, Workflows, performance, grants, dashboard, MLflow and Genie. Those are Days 3 to 5.

6A.10 Additions to your Day 1 evidence checklist

- [ ] `docs/01_BRS.md`, `02_HLD.md`, `03_LLD.md`, `04_ERD.md`, `05_data_dictionary.md`, `06_source_to_target.md`
- [ ] RAID tracker (inside the BRS file)
- [ ] Bronze contract test output (six PASS)
- [ ] Lineage screenshot
- [ ] PR checklist file (section 3A)

7. Day 2 – What Silver is and the rule catalogue

Silver is the version of the data you are willing to stand behind. For every raw row exactly one of these happens:

- **Reject** – the row cannot be trusted or used (for example no `flight_id`). It goes to `silver_rejected_records` with a rule ID, a reason and the source file, so nothing disappears without a trace.
- **Keep and flag** – the row is odd but may be real (an extreme fare). It stays in Silver with a true/false flag.
- **Keep and fill** – a missing optional value is replaced with a clear marker (`UNKNOWN`) and flagged.

The spec says the defects in this feed are 'not automatically invalid business transactions', so on *this* data almost everything is keep-and-flag. That is why most reject rules will count **zero** on the real files. Do not worry about that: the rules define the contract, and your unit tests (section 11) prove they fire when bad data arrives.

The catalogue

Create `docs/dq_rule_catalogue.md` with this table.

ID	Rule	Severity	Action	Expected on this data
R01	`flight_id` missing or not `FLIGHT-` plus 7 digits	Critical	Reject	0
R02	`event_timestamp` missing or not `yyyy-MM-ddTHH:mm:ss`	Critical	Reject	0
R03	`ingest_timestamp` missing or not that format	Critical	Reject	0
R04	`delay_minutes` missing or not an integer	Critical	Reject	0
R05	`delay_minutes` negative	Critical	Reject	0
R06	`fare_amount` missing or not numeric	Critical	Reject	0
R07	`airline_id` or `airport_id` missing	Critical	Reject	0
R08	`flight_status` not in OPEN, PENDING, COMPLETED, FAILED	Critical	Reject	0
R10	Exact duplicate (same `flight_id`, same 12 columns)	Medium	Drop copy, audit in rejected table	**40**
R11	Same `flight_id` with different content; older record loses	Medium	Drop older, audit	0
F01	`fare_amount` above 2,350	Warning	Keep, `is_fare_outlier = true`	**35**
F02	`delay_minutes` above 180	Warning	Keep, `is_delay_outlier = true`	0
F03	`flight_type` missing	Warning	Keep, fill `UNKNOWN`, `flight_type_was_missing = true`	**12**
F04	`timezone` is not a real time-zone name	Warning	Keep, assume UTC, `timezone_is_valid = false`	**11,500**
C01 to C05	CDC row: missing flight, bad timestamp, bad sequence, status not COMPLETED/REVERSED/CANCELLED, flight not in Silver	Critical	Reject (stage `CDC`)	0

Two more things to write in the log:

- **Tie-breaker (D05):** for exact duplicates the survivor is the copy from the earliest file name (`flight_batch_01` before `flight_batch_04`). For a changed version of the same flight (R11) the newest ingest time wins, and file name breaks any tie. The ordering is fully specified, so a rerun always keeps the same rows.
- **The counting rule that must always hold:** *Bronze rows = Silver rows + rejected rows.* For events that is `11,540 = 11,500 + 40`.

8. Day 2 – The functions notebook (`03_silver_functions`)

Put all logic in functions so the pipeline *and* the unit tests run the **same code**. Create the notebook `03_silver_functions`.

Cell 1 (alone in its cell):

```
%run ./00_config
```

Cell 2 – cleaning, dedupe, CDC and as-of functions. I ran this exact code against your CSVs:

```
import zoneinfo
from pyspark.sql import functions as F, Window

# ---------- constants (every one of these goes in your decision log) -------
---
BASE_COLS = ['flight_id', 'event_timestamp', 'ingest_timestamp',
'airline_id', 'airport_id',
             'flight_type', 'flight_status', 'timezone', 'fare_amount',
'delay_minutes',
             'aircraft_id', 'source_system']
TS_REGEX = '^[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}$'
ALLOWED_STATUS = ['OPEN', 'PENDING', 'COMPLETED', 'FAILED']
CDC_ALLOWED_STATUS = ['COMPLETED', 'REVERSED', 'CANCELLED']
FARE_OUTLIER_MAX = 2350.0      # Q3 + 3*IQR from profiling; normal max in the
data is 1,099.99
DELAY_OUTLIER_MAX = 180        # minutes; nothing in this dataset exceeds 90
DEFAULT_SOURCE_TZ = 'UTC'      # assumption: the timezone column holds USD,
which is not a time zone
VALID_TZ = sorted(zoneinfo.available_timezones())


def _ts(col_name):
    '''Strict ISO timestamp: must match the contract regex AND cast cleanly,
else NULL.'''
    return F.when(F.col(col_name).rlike(TS_REGEX),
F.expr(f'try_cast({col_name} as timestamp)'))
```

```
def classify_events(bronze_df):
    '''Parse, convert to UTC, add flags, and decide which (single) reject
rule fires first.'''
    d = bronze_df
    if 'source_schema_version' not in d.columns:      # batches 1-9 do not
have it
        d = d.withColumn('source_schema_version', F.lit(None).cast('string'))
    # trim text columns, treat blank as NULL (raw values stay untouched in
Bronze)
    for c in BASE_COLS:
        d = d.withColumn(c, F.nullif(F.trim(F.col(c)), F.lit('')))

    d = (d
         .withColumn('_row_hash', F.sha2(F.concat_ws('||', *
[F.coalesce(F.col(c), F.lit('<NULL>')) for c in BASE_COLS]), 256))
         .withColumn('event_ts_local', _ts('event_timestamp'))
         .withColumn('ingest_ts_local', _ts('ingest_timestamp'))
         .withColumn('delay_minutes_int', F.expr('try_cast(delay_minutes as
int)'))
         .withColumn('fare_amount_dec', F.expr('try_cast(fare_amount as
decimal(12,2))'))
         .withColumn('tz_is_valid',
F.coalesce(F.col('timezone').isin(VALID_TZ), F.lit(False)))
         .withColumn('tz_used', F.when(F.col('tz_is_valid'),
F.col('timezone')).otherwise(F.lit(DEFAULT_SOURCE_TZ)))
         .withColumn('event_ts_utc', F.to_utc_timestamp('event_ts_local',
F.col('tz_used')))
         .withColumn('ingest_ts_utc', F.to_utc_timestamp('ingest_ts_local',
F.col('tz_used')))
         )

    checks = [
        ('R01', 'flight_id missing or not FLIGHT-nnnnnnn',
         F.coalesce(~F.col('flight_id').rlike('^FLIGHT-[0-9]{7}$'),
F.lit(True))),
        ('R02', 'event_timestamp missing or not yyyy-MM-ddTHH:mm:ss',
F.col('event_ts_local').isNull()),
        ('R03', 'ingest_timestamp missing or not yyyy-MM-ddTHH:mm:ss',
F.col('ingest_ts_local').isNull()),
        ('R04', 'delay_minutes missing or not an integer',
F.col('delay_minutes_int').isNull()),
        ('R05', 'delay_minutes is negative', F.col('delay_minutes_int') < 0),
        ('R06', 'fare_amount missing or not numeric',
F.col('fare_amount_dec').isNull()),
```

```
        ('R07', 'airline_id or airport_id missing',
F.col('airline_id').isNull() | F.col('airport_id').isNull()),
        ('R08', 'flight_status missing or not in allowed list',
         F.coalesce(~F.col('flight_status').isin(ALLOWED_STATUS),
F.lit(True))),
    ]
    rule = F.lit(None).cast('string')
    reason = F.lit(None).cast('string')
    for rid, why, cond in reversed(checks):          # reversed so the FIRST
failing rule wins
        rule = F.when(cond, F.lit(rid)).otherwise(rule)
        reason = F.when(cond, F.lit(why)).otherwise(reason)
    return d.withColumn('reject_rule', rule).withColumn('reject_reason',
reason)


def split_and_dedupe(classified):
    '''Returns (silver_df, rejected_df). Rejected = invalid rows (R01-R08) +
removed duplicates (R10, R11).'''
    invalid = classified.filter(F.col('reject_rule').isNotNull())
    valid = classified.filter(F.col('reject_rule').isNull())

    # R10: same flight_id AND identical content -> keep the copy from the
earliest file
    w_exact = Window.partitionBy('flight_id',
'_row_hash').orderBy(F.col('_source_file').asc(),
F.col('_ingested_at').asc())
    valid = valid.withColumn('_rn_exact', F.row_number().over(w_exact))
    dup_exact = (valid.filter('_rn_exact > 1')
                 .withColumn('reject_rule', F.lit('R10'))
                 .withColumn('reject_reason', F.lit('Exact duplicate of an
earlier copy (same flight_id, same content)')))
    valid = valid.filter('_rn_exact = 1')

    # R11: same flight_id, different content -> latest ingest time wins; file
name breaks ties
    w_ver =
Window.partitionBy('flight_id').orderBy(F.col('ingest_ts_utc').desc(),
F.col('_source_file').asc())
    valid = valid.withColumn('_rn_ver', F.row_number().over(w_ver))
    dup_ver = (valid.filter('_rn_ver > 1')
               .withColumn('reject_rule', F.lit('R11'))
               .withColumn('reject_reason', F.lit('Older version of a
flight_id that has a newer record')))
    keep = valid.filter('_rn_ver = 1')
```

```
    silver = keep.select(
        'flight_id', 'event_ts_utc', 'ingest_ts_utc', 'airline_id',
'airport_id',
        F.coalesce(F.col('flight_type'),
F.lit('UNKNOWN')).alias('flight_type'),
        F.col('flight_type').isNull().alias('flight_type_was_missing'),
        'flight_status',
        F.col('timezone').alias('timezone_raw'), 'tz_used',
F.col('tz_is_valid').alias('timezone_is_valid'),
        F.col('fare_amount_dec').alias('fare_amount'),
        F.col('delay_minutes_int').alias('delay_minutes'),
        'aircraft_id', 'source_system', 'source_schema_version',
        (F.col('fare_amount_dec') >
F.lit(FARE_OUTLIER_MAX)).alias('is_fare_outlier'),
        (F.col('delay_minutes_int') >
F.lit(DELAY_OUTLIER_MAX)).alias('is_delay_outlier'),
        '_row_hash', '_source_file', '_ingested_at',
        F.current_timestamp().alias('_processed_at'))

    rejected = (invalid.unionByName(dup_exact, allowMissingColumns=True)
                .unionByName(dup_ver, allowMissingColumns=True)
                .select(F.lit('EVENTS').alias('stage'),
                        F.col('reject_rule').alias('rule_id'),
                        F.col('reject_reason').alias('reason'),
                        F.col('flight_id'),
                        F.col('_source_file').alias('source_file'),
                        F.to_json(F.struct(*BASE_COLS)).alias('raw_record'),
                        F.current_timestamp().alias('rejected_at')))
    return silver, rejected


# ---------------- CDC ----------------
def classify_cdc(bronze_cdc_df, known_flights_df):
    '''Parse CDC rows, convert to UTC, flag invalid ones. known_flights_df
has one column: flight_id.'''
    d = bronze_cdc_df
    for c in ['flight_id', 'change_timestamp', 'previous_status',
'new_status', 'change_reason', 'source_sequence']:
        d = d.withColumn(c, F.nullif(F.trim(F.col(c)), F.lit('')))
    d = (d.withColumn('change_ts_local', _ts('change_timestamp'))
          .withColumn('change_ts_utc', F.to_utc_timestamp('change_ts_local',
F.lit(DEFAULT_SOURCE_TZ)))
          .withColumn('source_sequence_n', F.expr('try_cast(source_sequence
as bigint)'))

.join(F.broadcast(known_flights_df.select('flight_id').distinct().withColumn(
```

```
'_known', F.lit(True))),
                'flight_id', 'left'))
    checks = [
        ('C01', 'flight_id missing', F.col('flight_id').isNull()),
        ('C02', 'change_timestamp missing or not yyyy-MM-ddTHH:mm:ss',
F.col('change_ts_local').isNull()),
        ('C03', 'source_sequence missing or not an integer',
F.col('source_sequence_n').isNull()),
        ('C04', 'new_status not in COMPLETED/REVERSED/CANCELLED',
         F.coalesce(~F.col('new_status').isin(CDC_ALLOWED_STATUS),
F.lit(True))),
        ('C05', 'flight_id not found in silver_flight_current (orphan
update)', F.col('_known').isNull()),
    ]
    rule = F.lit(None).cast('string'); reason = F.lit(None).cast('string')
    for rid, why, cond in reversed(checks):
        rule = F.when(cond, F.lit(rid)).otherwise(rule)
        reason = F.when(cond, F.lit(why)).otherwise(reason)
    return d.withColumn('reject_rule', rule).withColumn('reject_reason',
reason)


def latest_change_per_flight(valid_cdc):
    '''MERGE needs exactly one source row per key: keep the newest change
(time, then sequence).'''
    w =
Window.partitionBy('flight_id').orderBy(F.col('change_ts_utc').desc(),
F.col('source_sequence_n').desc())
    return (valid_cdc.withColumn('_rn', F.row_number().over(w)).filter('_rn =
1')
            .select('flight_id', 'new_status', 'change_ts_utc',
F.col('source_sequence_n').alias('source_sequence')))


def status_as_of(events_df, history_df):
    '''Status in effect at each event time = newest history row with
effective_ts_utc <= event_ts_utc.'''
    e = events_df.select('flight_id', 'event_ts_utc').alias('e')
    h = history_df.select('flight_id', 'effective_ts_utc', 'new_status',
'source_sequence').alias('h')
    j = e.join(h, (F.col('e.flight_id') == F.col('h.flight_id')) &
(F.col('h.effective_ts_utc') <= F.col('e.event_ts_utc')), 'left')
    w = Window.partitionBy('e.flight_id',
'e.event_ts_utc').orderBy(F.col('h.effective_ts_utc').desc(),
F.col('h.source_sequence').desc())
    return (j.withColumn('_rn', F.row_number().over(w)).filter('_rn = 1')
```

```
            .select(F.col('e.flight_id').alias('flight_id'),
F.col('e.event_ts_utc').alias('event_ts_utc'),
                    F.col('h.new_status').alias('status_at_event_time'),

F.col('h.effective_ts_utc').alias('status_effective_ts_utc')))
```

Cell 3 – helpers that write to Delta tables (this part only runs on Databricks):

```
def ensure_tables():
    spark.sql(f'''CREATE TABLE IF NOT EXISTS {t('silver_rejected_records')} (
        stage STRING, rule_id STRING, reason STRING, flight_id STRING,
        source_file STRING, raw_record STRING, rejected_at TIMESTAMP) USING
DELTA''')
    spark.sql(f'''CREATE TABLE IF NOT EXISTS {t('silver_flight_current')} (
        flight_id STRING, airline_id STRING, airport_id STRING, flight_type
STRING,
        event_ts_utc TIMESTAMP, fare_amount DECIMAL(12,2), delay_minutes INT,
is_fare_outlier BOOLEAN,
        initial_status STRING, current_status STRING, status_updated_at
TIMESTAMP,
        last_source_sequence BIGINT, status_source STRING) USING DELTA''')
    spark.sql(f'''CREATE TABLE IF NOT EXISTS {t('silver_status_history')} (
        flight_id STRING, effective_ts_utc TIMESTAMP, previous_status STRING,
new_status STRING,
        change_reason STRING, source_sequence BIGINT, record_type STRING,
is_noop BOOLEAN,
        source_file STRING, processed_at TIMESTAMP) USING DELTA''')


def replace_rejected(stage, rejected_df):
    '''Idempotent: wipe this stage rejects, then write the fresh ones.'''
    spark.sql(f'''DELETE FROM {t('silver_rejected_records')} WHERE stage =
'{stage}' ''')
    (rejected_df.select('stage', 'rule_id', 'reason', 'flight_id',
'source_file', 'raw_record', 'rejected_at')
        .write.mode('append').saveAsTable(t('silver_rejected_records')))


def seed_current(target, events_df):
    '''Insert flights that are not in the current table yet. Never touches
existing rows,
    so re-running Silver does not erase status changes already applied by
CDC.'''
    src = events_df.select('flight_id', 'airline_id', 'airport_id',
'flight_type', 'event_ts_utc', 'fare_amount',
                           'delay_minutes', 'is_fare_outlier',
```

```
                            F.col('flight_status').alias('initial_status'),
                            F.col('flight_status').alias('current_status'),
                            F.col('event_ts_utc').alias('status_updated_at'),

F.lit(0).cast('bigint').alias('last_source_sequence'),
                            F.lit('EVENT').alias('status_source'))
    src.createOrReplaceTempView('_seed_src')
    spark.sql(f'MERGE INTO {target} t USING _seed_src s ON t.flight_id =
s.flight_id '
              f'WHEN NOT MATCHED THEN INSERT *')


def append_history(target, history_df):
    '''Insert-only MERGE on (flight_id, source_sequence): replaying the same
rows adds nothing.'''
    history_df.createOrReplaceTempView('_hist_src')
    spark.sql(f'MERGE INTO {target} t USING _hist_src s '
              f'ON t.flight_id = s.flight_id AND t.source_sequence =
s.source_sequence '
              f'WHEN NOT MATCHED THEN INSERT *')


def merge_cdc(target, latest_df):
    '''Apply the newest change per flight, but only if it is newer than what
the row already has.'''
    latest_df.createOrReplaceTempView('_cdc_latest')
    spark.sql(f'''
        MERGE INTO {target} t USING _cdc_latest s ON t.flight_id =
s.flight_id
        WHEN MATCHED AND (s.change_ts_utc > t.status_updated_at
                          OR (s.change_ts_utc = t.status_updated_at AND
s.source_sequence > t.last_source_sequence))
        THEN UPDATE SET t.current_status = s.new_status, t.status_updated_at
= s.change_ts_utc,
                        t.last_source_sequence = s.source_sequence,
t.status_source = 'CDC' ''')
```

Run both cells. Expected result: no output and no error. If you see `NameError: spark`, the notebook is not attached to serverless compute.

How the functions work (read this once; you will be asked in the walkthrough)

- **`try_cast` and the timestamp regex.** A normal `cast` can crash the whole job on one bad value. `try_cast` returns NULL instead, and the rule then sends that row to the rejected table.

- **Blank = NULL.** CSV blanks are trimmed and turned into NULL, so one rule handles both.
- **First failing rule wins.** A row with two problems is rejected once, under the first rule it breaks, so the counting equation stays exact.
- **`_row_hash`** is a fingerprint of the 12 base columns only. That is how the copy in a batch-10 file (with the extra column) is still recognised as a duplicate of the copy in an older file.
- **Time zones.** If `timezone` is a real name such as `Asia/Kolkata`, the time is converted to UTC. If not (your real data: `USD`), the code assumes UTC and sets `timezone_is_valid = false`. The assumption lives in one constant, `DEFAULT_SOURCE_TZ`, so changing your mind is a one-line edit.
- **MERGE guard.** `merge_cdc` only overwrites a status when the incoming change is newer (or has the same time and a higher sequence). That single condition is what makes late updates and replays safe.
- **`seed_current` never updates.** It only inserts flights that are new, so rebuilding Silver events cannot wipe out statuses already changed by CDC.

8.1 Line-by-line walk-through of the code

You will have to explain this code to a client, so here it is in order. Follow **FLIGHT-0000501**, which appears twice in your data: in `flight_batch_01.csv` and in `flight_batch_04.csv`.

`classify_events` – turns raw text into typed, checked values

1. *Add `source_schema_version` if missing.* Batches 1 to 9 do not have that column. Adding an empty one means later code can always rely on it.
2. *The loop over `BASE_COLS`.* Each of the 12 columns is trimmed, and an empty string becomes NULL. After this a blank `flight_type` is a real NULL.
3. *`_row_hash`.* The 12 values are joined into one string with `||` between them (a NULL becomes the marker `<NULL>`), then SHA-256 turns that into a fingerprint. Two rows with the same fingerprint have identical content. Both copies of FLIGHT-0000501 get the same fingerprint.
4. *`event_ts_local` and `ingest_ts_local`.* `_ts` only converts when the text matches the format `yyyy-MM-ddTHH:mm:ss`, using `try_cast`, which gives NULL instead of crashing. `2026-07-02T09:22:00` becomes a real timestamp; `yesterday` becomes NULL.
5. *`delay_minutes_int`, `fare_amount_dec`.* `try_cast` again: `77` becomes the number 77, `abc` becomes NULL.

6. *`tz_is_valid`, `tz_used`.* If `timezone` is a real zone name from Python's zone list it is used; otherwise `UTC`. `USD` is not a zone name, so `tz_is_valid` is false and `tz_used` is `UTC`.
7. *`event_ts_utc`.* `to_utc_timestamp` shifts the time from `tz_used` into UTC. With `UTC` the value is unchanged.
8. *The `checks` list and the `reversed` loop.* Each check is a rule ID, a reason and a true-when-bad condition. Looping in reverse and wrapping each one around the previous builds a chain equal to 'if R01 fails use R01, else if R02 fails use R02, … else no rule'. Reversing makes the first rule in the list win. That is why a row is rejected only once.

`split_and_dedupe` – removes duplicates and builds the outputs

9. *Split.* Rows with a rule ID are `invalid`; the rest are `valid`.
10. *R10 window.* Rows are grouped by `flight_id` and fingerprint and numbered 1, 2, … ordered by file name, then load time. Copy A (batch 01) gets 1 and survives; copy B (batch 04) gets 2 and becomes an R10 reject.
11. *R11 window.* Same idea by `flight_id` only, newest ingest time first. It finds nothing in this data but protects you if a changed version ever arrives.
12. *The Silver `select`.* Renames columns, fills `flight_type` with `UNKNOWN`, adds the two outlier flags by comparing with the constants, and stamps `_processed_at`.
13. *The rejected output.* Union of invalid rows and both kinds of removed duplicate, each with `to_json(struct(...))`, a copy of the original row as JSON so you can always see exactly what was rejected.

Step	Copy A (batch 01)	Copy B (batch 04)
Raw columns	identical	identical
Fingerprint	same	same
R10 position	1	2
Result	kept in `silver_flight_events`	rejected as R10

The CDC functions

14. `classify_cdc` repeats the same pattern for status updates and adds one check: a left join to the known flights, and a flight that did not match is an orphan (C05).
15. `latest_change_per_flight` numbers each flight's changes newest first (time, then sequence) and keeps number 1, so MERGE sees one row per flight.
16. `status_as_of` joins every history row at or before the event time to the event, numbers them newest first and keeps number 1: the status in effect at that moment.

17. `merge_cdc` updates a row only when the incoming change is newer than the stored one. That single condition is the whole defence against late data.

9. Day 2 – Build `silver_flight_events` (notebook `04_silver_events`)

Cell 1 (alone in its cell):

```
%run ./03_silver_functions
```

Cell 2 – build it:

```
ensure_tables()

bronze = spark.table(t('bronze_flight_events'))
classified = classify_events(bronze)
silver_df, rejected_df = split_and_dedupe(classified)

(silver_df.write.mode('overwrite').option('overwriteSchema', 'true')
    .saveAsTable(t('silver_flight_events')))
replace_rejected('EVENTS', rejected_df)

print('silver_flight_events:',
spark.table(t('silver_flight_events')).count())
print('rejected (EVENTS):',
spark.table(t('silver_rejected_records')).filter(F.col('stage') ==
'EVENTS').count())
```

Expected result: `silver_flight_events: 11500` and `rejected (EVENTS): 40`.

Add to your decision log: **D13** – *`silver_flight_events` is rebuilt from Bronze on every run.* It is simple and gives the same result every time (idempotent). With millions of rows you would process only new Bronze rows instead; at 11,540 rows a rebuild takes seconds.

9.1 Verify (these outputs are your Silver evidence)

Run each as its own cell.

```
# 1. Nothing lost: Bronze = Silver + rejected
b = spark.table(t('bronze_flight_events')).count()
s = spark.table(t('silver_flight_events')).count()
r = spark.table(t('silver_rejected_records')).filter(F.col('stage') ==
'EVENTS').count()
print(b, '=', s, '+', r, '->', b == s + r)
assert b == s + r
```

Expected: `11540 = 11500 + 40 -> True`

```
# 2. Acceptance condition: no duplicate business keys in the curated table
dup_keys =
spark.table(t('silver_flight_events')).groupBy('flight_id').count().filter('c
ount > 1').count()
print('duplicate flight_id in silver:', dup_keys)
assert dup_keys == 0
```

Expected: `0`

```
# 3. What was rejected and why
display(spark.table(t('silver_rejected_records')).groupBy('stage', 'rule_id',
'reason').count())
```

Expected: one row: `EVENTS`, `R10`, 'Exact duplicate of an earlier copy…', `40`.

```
# 4. Flag counts
display(spark.table(t('silver_flight_events')).agg(
    F.sum(F.col('is_fare_outlier').cast('int')).alias('fare_outliers'),
    F.sum(F.col('is_delay_outlier').cast('int')).alias('delay_outliers'),

F.sum(F.col('flight_type_was_missing').cast('int')).alias('flight_type_filled
'),

F.sum((~F.col('timezone_is_valid')).cast('int')).alias('timezone_invalid')))
```

Expected: `35`, `0`, `12`, `11500`. These match the manifest: 35 injected outliers, 12 optional nulls. The 40 duplicates did not contain any of those, which is why the numbers stay the same after dedupe.

```
# 5. One duplicate traced end to end
display(spark.table(t('silver_flight_events')).filter(F.col('flight_id') ==
'FLIGHT-0000501')
        .select('flight_id', '_source_file', 'event_ts_utc', 'fare_amount',
'delay_minutes'))
display(spark.table(t('silver_rejected_records')).filter(F.col('flight_id')
== 'FLIGHT-0000501'))
```

Expected: Silver keeps the copy from `flight_batch_01.csv`. The rejected table holds the copy from `flight_batch_04.csv`, rule `R10`, with the original row as JSON in `raw_record`. This is a perfect example for the client walkthrough.

```
# 6. Types and status mix
spark.table(t('silver_flight_events')).printSchema()
display(spark.table(t('silver_flight_events')).groupBy('flight_status').count
().orderBy('flight_status'))
```

Expected: `event_ts_utc` is `timestamp`, `fare_amount` is `decimal(12,2)`, `delay_minutes` is `integer`. Status counts: COMPLETED 2,848 · FAILED 2,968 · OPEN 2,870 · PENDING 2,814 (total 11,500).

Replay check (TC03 for Silver): run Cell 2 again and repeat checks 1 and 2. The counts must be identical. Screenshot both runs.

Before / after table for your evidence pack:

Stage	Rows
Bronze `bronze_flight_events`	11,540
Removed as exact duplicates (R10)	40
Rejected as invalid (R01 to R08)	0
Silver `silver_flight_events`	11,500
Of which flagged: fare outlier / filled flight type / invalid time zone	35 / 12 / 11,500

10. Day 2 – Status CDC, history, UTC and the as-of join (notebook `05_silver_cdc`)

10.1 The idea in plain words

CDC (change data capture) means the source does not resend whole flights, it sends *changes*: 'flight X moved from PENDING to CANCELLED at this time'. You keep two things:

- **`silver_flight_current`** – one row per flight with its latest status. Dashboards read this.
- **`silver_status_history`** – every status the flight ever had, including the original one from the flight event. Nothing is overwritten here, only added.

The danger is applying an *old* change after a *newer* one has already been applied. The guard in `merge_cdc` prevents that. Your real file has no late or out-of-order rows, so you will prove the guard with unit tests in section 11.

Cell 1 (alone in its cell): `%run ./03_silver_functions`

Cell 2 – create the current-state table from Silver events:

```
ensure_tables()
cur = t('silver_flight_current')
seed_current(cur, spark.table(t('silver_flight_events')))
print('silver_flight_current rows:', spark.table(cur).count())
display(spark.table(cur).groupBy('status_source').count())
```

Expected (first run): `11500`, and `status_source = EVENT` for all 11,500 rows. No CDC has been applied yet.

Cell 3 – validate the CDC rows and quarantine bad ones:

```
cdc = classify_cdc(spark.table(t('bronze_status_cdc')),
spark.table(cur).select('flight_id'))

cdc_rejected = (cdc.filter(F.col('reject_rule').isNotNull())
    .select(F.lit('CDC').alias('stage'),
F.col('reject_rule').alias('rule_id'),
F.col('reject_reason').alias('reason'),
            'flight_id', F.col('_source_file').alias('source_file'),
            F.to_json(F.struct('flight_id', 'change_timestamp',
'previous_status', 'new_status',
                               'change_reason',
'source_sequence')).alias('raw_record'),
            F.current_timestamp().alias('rejected_at')))
replace_rejected('CDC', cdc_rejected)

cdc_valid = cdc.filter(F.col('reject_rule').isNull())
print('CDC rows:', cdc.count(), '| rejected:', cdc_rejected.count(), '|
valid:', cdc_valid.count())
```

Expected: `CDC rows: 520 | rejected: 0 | valid: 520`

Cell 4 – write the history (original status first, then every change):

```
initial_hist = (spark.table(t('silver_flight_events'))
    .select('flight_id', F.col('event_ts_utc').alias('effective_ts_utc'),
            F.lit(None).cast('string').alias('previous_status'),
F.col('flight_status').alias('new_status'),
            F.lit('INITIAL_EVENT').alias('change_reason'),
F.lit(0).cast('bigint').alias('source_sequence'),
            F.lit('INITIAL').alias('record_type'),
F.lit(False).alias('is_noop'),
            F.col('_source_file').alias('source_file'),
F.current_timestamp().alias('processed_at')))

cdc_hist = (cdc_valid
    .select('flight_id', F.col('change_ts_utc').alias('effective_ts_utc'),
'previous_status', 'new_status', 'change_reason',
            F.col('source_sequence_n').alias('source_sequence'),
F.lit('CDC').alias('record_type'),
            (F.col('previous_status') ==
F.col('new_status')).alias('is_noop'),
            F.col('_source_file').alias('source_file'),
F.current_timestamp().alias('processed_at')))

append_history(t('silver_status_history'),
initial_hist.unionByName(cdc_hist))
print('history rows:', spark.table(t('silver_status_history')).count())
```

Expected: `history rows: 12020` (11,500 original statuses + 520 changes).

Cell 5 – apply the changes to the current-state table:

```
latest = latest_change_per_flight(cdc_valid)
print('flights with a change to apply:', latest.count())
merge_cdc(cur, latest)

display(spark.sql(f'DESCRIBE HISTORY {cur}')
        .select('version', 'operation', F.col('operationMetrics')
['numTargetRowsUpdated'].alias('rows_updated')))
display(spark.table(cur).groupBy('current_status').count().orderBy('current_s
tatus'))
display(spark.table(cur).groupBy('status_source').count())
```

Expected (first run):

Check	Value
flights with a change to apply	520

Check	Value
latest `MERGE` shows `rows_updated`	520
`current_status`	CANCELLED 171 · COMPLETED 2,890 · FAILED 2,836 · OPEN 2,740 · PENDING 2,690 · REVERSED 173 (total 11,500)
`status_source`	EVENT 10,980 · CDC 520

The 2,890 comes from 2,714 flights that were already COMPLETED plus 176 that the CDC moved to COMPLETED. The earlier statuses shrink accordingly.

Acceptance check (run as its own cell):

```
n = spark.table(cur).count()
d = spark.table(cur).select('flight_id').distinct().count()
print(n, d)
assert n == d == 11500
```

Replay test (TC03 and TC04 evidence). Run Cells 3, 4 and 5 a second time. Expected: `history rows` is still `12020`; the status counts above are **unchanged**; and `DESCRIBE HISTORY` either shows a new MERGE with `rows_updated = 0` or no new entry at all. Either one proves a replay changes nothing. Screenshot it.

10.2 Follow one flight from raw to Silver (use this in your walkthrough)

```
fid = 'FLIGHT-0000001'
display(spark.table(t('bronze_status_cdc')).filter(F.col('flight_id') ==
fid))
display(spark.table(t('silver_status_history')).filter(F.col('flight_id') ==
fid).orderBy('effective_ts_utc', 'source_sequence'))
display(spark.table(cur).filter(F.col('flight_id') == fid))
```

Expected: the raw CDC row says PENDING → CANCELLED, sequence 1, `2026-08-10T00:00:00`. History has two rows: `INITIAL` PENDING at `2026-07-01 00:02:00` and `CDC` CANCELLED at `2026-08-10 00:00:00`. The current row says `CANCELLED`, `status_source = CDC`.

10.3 The domain rule: UTC and the 'temporal join'

The spec asks you to match flight events to schedule revisions by effective timestamp and to convert local times to UTC. Be honest about what the data allows:

- **UTC.** The `timezone` column is `USD` everywhere, so there is no local zone to convert from. Decision D08: treat timestamps as already UTC, flag it (`timezone_is_valid = false`), list it as a known limitation. The code *does* convert correctly when a real zone name is present; test U01 proves it (10:00 `Asia/Kolkata` becomes 04:30 UTC).
- **Revisions.** There is no schedule table in the pack. The only revision stream is `status_updates` , so the as-of join runs on status history (D09): for each flight event, find the newest history row with an effective time at or before the event time.

```
asof = status_as_of(spark.table(t('silver_flight_events')),
spark.table(t('silver_status_history')))
(asof.write.mode('overwrite').option('overwriteSchema', 'true')
     .saveAsTable(t('silver_flight_event_status_asof')))

chk = (spark.table(t('silver_flight_event_status_asof'))
       .join(spark.table(t('silver_flight_events')).select('flight_id',
'flight_status'), 'flight_id'))
print('rows:', chk.count(), '| status at event time equals reported status:',
      chk.filter('status_at_event_time = flight_status').count())
```

Expected: `rows: 11500 | status at event time equals reported status: 11500` .

Why is that 100%? Every CDC change happened on 2026-08-10, nine days after the last flight event (2026-08-01). So at the moment of every event, the status was still the one the event reported. The join is correct, but this dataset cannot show it doing anything interesting; test U10 does, using a made-up case where a change comes *before* the event. State that clearly in the walkthrough rather than claiming more.

11. Day 2 – Unit tests (notebook `06_tests_silver`)

Run this notebook **after** `05_silver_cdc` (two tests use temporary Delta tables copied from your real ones). The tests call the same functions as the pipeline.

Cell 1: `%run ./03_silver_functions`

Cell 2 – helpers:

```
def ev(**kw):
    '''One valid raw flight row; override any field to make it bad.'''
    base = dict(flight_id='FLIGHT-0000001', event_timestamp='2026-07-
01T10:00:00', ingest_timestamp='2026-07-01T10:05:00',
                airline_id='AIRLINE_ID-0001', airport_id='AIRPORT_ID-0001',
flight_type='DOMESTIC', flight_status='OPEN',
                timezone='USD', fare_amount='500.00', delay_minutes='20',
aircraft_id='AIRCRAFT_ID-0001',
                source_system='ADLS_EVENT', source_schema_version=None,
```

```
_source_file='file_a.csv')
    base.update(kw)
    return base


def mk(rows):
    schema = ', '.join(f'{k} string' for k in rows[0].keys())
    return spark.createDataFrame(rows, schema).withColumn('_ingested_at',
F.current_timestamp())


def run(rows):
    return split_and_dedupe(classify_events(mk(rows)))


CDC_SCHEMA = ('flight_id string, change_timestamp string, previous_status
string, new_status string, '
              'change_reason string, source_sequence string')


def cdc_rows(rows):
    return spark.createDataFrame(rows, CDC_SCHEMA).withColumn('_source_file',
F.lit('cdc_test.csv'))


results = []


def check(test_id, description, condition):
    results.append((test_id, description, 'PASS' if condition else 'FAIL'))
```

Cell 3 – cleaning tests (no Delta needed):

```
check('U00', 'time-zone list is available on this compute', 'Asia/Kolkata' in
VALID_TZ)

# U01 valid row + real time zone converts to UTC
s, r = run([ev(timezone='Asia/Kolkata')])
check('U01', 'valid row kept; 10:00 Asia/Kolkata becomes 04:30 UTC',
      r.count() == 0 and str(s.first().event_ts_utc) == '2026-07-01
04:30:00')

# U02 every reject rule routes to the right rule ID
bad = [ev(flight_id=None),
       ev(flight_id='FLIGHT-0000002', event_timestamp='yesterday'),
       ev(flight_id='FLIGHT-0000003', ingest_timestamp='2026-13-
```

```
45T99:00:00'),
       ev(flight_id='FLIGHT-0000004', delay_minutes='abc'),
       ev(flight_id='FLIGHT-0000005', delay_minutes='-5'),
       ev(flight_id='FLIGHT-0000006', fare_amount='N/A'),
       ev(flight_id='FLIGHT-0000007', airport_id=None),
       ev(flight_id='FLIGHT-0000008', flight_status='LANDED'),
       ev(flight_id='BADKEY')]
s, r = run(bad)
got = {x.flight_id: x.rule_id for x in r.collect()}
expected = {None: 'R01', 'FLIGHT-0000002': 'R02', 'FLIGHT-0000003': 'R03',
'FLIGHT-0000004': 'R04',
            'FLIGHT-0000005': 'R05', 'FLIGHT-0000006': 'R06', 'FLIGHT-
0000007': 'R07',
            'FLIGHT-0000008': 'R08', 'BADKEY': 'R01'}
check('U02', 'malformed rows are rejected with the correct rule ID and none
reach Silver',
      s.count() == 0 and got == expected)

# U03 duplicates: exact copies (R10) and a changed version (R11)
rows = [ev(flight_id='FLIGHT-0000010', _source_file='file_b.csv'),
        ev(flight_id='FLIGHT-0000010', _source_file='file_a.csv'),
        ev(flight_id='FLIGHT-0000011'),
        ev(flight_id='FLIGHT-0000011', delay_minutes='99',
ingest_timestamp='2026-07-01T11:00:00')]
s, r = run(rows)
check('U03a', 'exact duplicate: copy from the earliest file survives, other
copy is R10',
      s.filter(F.col('flight_id') == 'FLIGHT-0000010').first()._source_file
== 'file_a.csv'
      and [x.rule_id for x in r.filter(F.col('flight_id') == 'FLIGHT-
0000010').collect()] == ['R10'])
check('U03b', 'changed version: newest ingest time survives, older is R11',
      s.filter(F.col('flight_id') == 'FLIGHT-0000011').first().delay_minutes
== 99
      and [x.rule_id for x in r.filter(F.col('flight_id') == 'FLIGHT-
0000011').collect()] == ['R11'])

# U04-U06 flags and fills keep the row
s, r = run([ev(timezone='USD', flight_type='', fare_amount='77083.50')])
x = s.first()
check('U04', 'USD is not a time zone: assume UTC and flag it', x.tz_used ==
'UTC' and x.timezone_is_valid is False)
check('U05', 'blank flight_type becomes UNKNOWN and is flagged',
x.flight_type == 'UNKNOWN' and x.flight_type_was_missing)
check('U06', 'extreme fare is kept and flagged, not rejected',
x.is_fare_outlier and r.count() == 0)
```

Cell 4 – CDC rules, ordering and the as-of join (no Delta needed):

```
known = run([ev()])[0].select('flight_id')

# U07 CDC validation rules
cdc_test = cdc_rows([
    ('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'CANCELLED', 'X', '2'),
    ('FLIGHT-9999999', '2026-08-10T00:05:00', 'OPEN', 'CANCELLED', 'X', '3'),
    ('FLIGHT-0000001', 'not a time', 'OPEN', 'CANCELLED', 'X', '4'),
    ('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'LANDED', 'X', '5'),
    ('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'CANCELLED', 'X',
'four')])
c = classify_cdc(cdc_test, known)
got = {x.source_sequence: x.reject_rule for x in c.collect()}
check('U07', 'CDC: valid row passes; orphan, bad time, bad status, bad
sequence are rejected',
      got == {'2': None, '3': 'C05', '4': 'C02', '5': 'C04', 'four': 'C03'})

# U08 out-of-order arrival inside one batch: newest time wins; equal time ->
higher sequence wins
batch = classify_cdc(cdc_rows([
    ('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'CANCELLED', 'X', '2'),
    ('FLIGHT-0000001', '2026-08-10T00:01:00', 'OPEN', 'COMPLETED', 'X', '1'),
    ('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'REVERSED', 'X',
'9')]), known)
lat = latest_change_per_flight(batch.filter(F.col('reject_rule').isNull()))
check('U08', 'out-of-order batch: latest change per flight is chosen
deterministically',
      lat.count() == 1 and lat.first().new_status == 'REVERSED' and
lat.first().source_sequence == 9)

# U10 as-of join: a change BEFORE the event time must win; a change AFTER
must not
ev_df = spark.createDataFrame([('F1', '2026-07-01 12:00:00')], 'flight_id
string, e string') \
             .withColumn('event_ts_utc', F.col('e').cast('timestamp'))
hist_df = spark.createDataFrame(
    [('F1', '2026-07-01 08:00:00', 'OPEN', 0), ('F1', '2026-07-01 10:00:00',
'CANCELLED', 5),
     ('F1', '2026-07-01 14:00:00', 'COMPLETED', 9)],
    'flight_id string, t string, new_status string, source_sequence bigint')
\
    .withColumn('effective_ts_utc', F.col('t').cast('timestamp'))
```

```
check('U10', 'as-of join returns the status in effect at event time
(CANCELLED, not OPEN or COMPLETED)',
      status_as_of(ev_df, hist_df).first().status_at_event_time ==
'CANCELLED')
```

Cell 5 – the MERGE tests (Delta, temporary tables):

```
test_cur = t('tmp_test_current')
test_hist = t('tmp_test_history')
real_cur = t('silver_flight_current')
real_hist = t('silver_status_history')
for name in [test_cur, test_hist]:
    spark.sql(f'DROP TABLE IF EXISTS {name}')
spark.sql(f'CREATE TABLE {test_cur} AS SELECT * FROM {real_cur} WHERE 1 = 0')
spark.sql(f'CREATE TABLE {test_hist} AS SELECT * FROM {real_hist} WHERE 1 =
0')

# one flight, status OPEN as of 2026-07-01 10:00 UTC
seed_current(test_cur, run([ev(flight_status='OPEN')])[0])


def apply_cdc(rows):
    c = classify_cdc(cdc_rows(rows),
spark.table(test_cur).select('flight_id'))
    merge_cdc(test_cur,
latest_change_per_flight(c.filter(F.col('reject_rule').isNull())))


def status_now():
    return spark.table(test_cur).first().current_status


newer = [('FLIGHT-0000001', '2026-08-10T00:05:00', 'OPEN', 'CANCELLED', 'X',
'2')]
late = [('FLIGHT-0000001', '2026-08-10T00:01:00', 'OPEN', 'COMPLETED', 'X',
'1')]

apply_cdc(newer)
check('U11a', 'a later status revision is applied', status_now() ==
'CANCELLED')

apply_cdc(late)
check('U11b', 'a LATE (older) update does not overwrite the newer status',
status_now() == 'CANCELLED')

before = spark.table(test_cur).collect()
```

```
apply_cdc(newer)
apply_cdc(late)
check('U12', 'replaying the same updates changes nothing (idempotent)',
spark.table(test_cur).collect() == before)

h = (spark.createDataFrame(
        [('FLIGHT-0000001', '2026-08-10 00:05:00', 'OPEN', 'CANCELLED', 'X',
2, 'CDC', False, 'cdc_test.csv'),
         ('FLIGHT-0000001', '2026-08-10 00:01:00', 'OPEN', 'COMPLETED', 'X',
1, 'CDC', False, 'cdc_test.csv')],
        'flight_id string, ts string, previous_status string, new_status
string, change_reason string, '
        'source_sequence bigint, record_type string, is_noop boolean,
source_file string')
     .withColumn('effective_ts_utc',
F.col('ts').cast('timestamp')).drop('ts')
     .withColumn('processed_at', F.current_timestamp()))
append_history(test_hist, h)
append_history(test_hist, h)
check('U13', 'history keeps the late update too, and a replay adds no
duplicate rows',
      spark.table(test_hist).count() == 2)

for name in [test_cur, test_hist]:
    spark.sql(f'DROP TABLE IF EXISTS {name}')
```

Cell 6 – the result table (screenshot this):

```
out = spark.createDataFrame(results, 'test_id string, description string,
result string').orderBy('test_id')
display(out)
failed = [r for r in results if r[2] != 'PASS']
assert not failed, f'{len(failed)} test(s) failed: {failed}'
print('ALL', len(results), 'TESTS PASSED')
```

Expected: `ALL 15 TESTS PASSED`, and every row in the result table says PASS. (There is no U09; the numbering skips it.)

What I verified here: I ran the cleaning, CDC-validation, ordering and as-of tests (U00 to U08 and U10, which is 11 checks) against a local Spark using the exact code above, and all 11 pass. The four MERGE checks (U11a, U11b, U12, U13) need Delta, so you are the first to run them. If one fails, section 12 explains the usual causes.

Spec test	Covered by
TC03 replay unchanged batch	Bronze replay (6.4), Silver rerun (9.1), CDC replay (10.1), U12, U13
TC04 later status revision	U11a, U11b, U08, history trace (10.2)
TC05 malformed input rejected with reason	U02, U07 (the full workflow-level test comes on Day 4)

12. Troubleshooting

What you see	Likely cause	Fix
`Catalog 'workspace' not found` or permission error creating the schema	Your catalog has another name or you lack rights	Run `SHOW CATALOGS`; change `CATALOG` in `00_config`
`Path does not exist: /Volumes/...`	The upload went to a different folder, or the volume/schema name differs	In Catalog Explorer open the volume; the path shown there must match `BASE` and `LANDING`; re-run the verify cell from step 1.5
`NameError: spark` or `dbutils`	Notebook is not attached to compute	Attach it to serverless compute
`%run` says notebook not found	Notebooks are in different folders, or `%run` shares a cell with other code	Keep all notebooks in the same folder; `%run` must be alone in its cell
Round 1 loads 0 rows	Files were never copied to the live folder, or the stream already processed them	Run `copy_to_live(...)`; check `dbutils.fs.ls(BASE + '/primary_events')`
Round 2 fails with an unknown-field or new-column message	Expected schema drift	Re-run the same cell once (see 6.3)

What you see	Likely cause	Fix
Delta error about a schema mismatch when writing	The table was created without `mergeSchema` or from an older run	Remove append-only (`SET TBLPROPERTIES ('delta.appendOnly' = 'false')`), then `reset_bronze('bronze_flight_events')` and redo rounds 1 and 2
Bronze count is double (23,080)	You deleted the table but not the checkpoint, or the reverse	Always use `reset_bronze(...)` , which removes both
`UPDATE` on Bronze succeeds	Append-only not set	Run step 6.7
`Cannot resolve try_cast` or `nullif` is not a member of `functions`	Older runtime	Use serverless compute. If you must use an older runtime, replace `F.nullif(x, F.lit(''))` with `F.when(F.trim(x) == '', None).otherwise(F.trim(x))`
MERGE fails: multiple source rows matched one target row	The source has two rows for one key	`seed_current` : Silver events must have unique `flight_id` (check 9.1 test 2). History: two rows share `flight_id` and `source_sequence` ; CDC sequences must be unique
MERGE `INSERT *` fails on column mismatch	A table was created earlier with a different layout	`DROP TABLE` the Silver table and run `ensure_tables()` again; for the temp tables just re-run Cell 5
Silver rows are not 11,500	Bronze is not 11,540, or dedupe logic was edited	Check the Bronze audit first, then compare your function cell with section 8
`U00` or `U01` fails	The time-zone list is empty on that compute	Print `len(VALID_TZ)` . If it is 0, tell me and I will give you a fixed fallback list
Everything is slow or stops	Compute is starting or was auto-terminated, or a quota or policy applies	Restart the compute or ask your admin about the quota; your data is kept

12A. Enterprise-workspace problems

What you see	Likely cause	Fix
`PERMISSION_DENIED` on `USE CATALOG` or `CREATE SCHEMA`	You lack privileges on that catalog	Use another catalog or send the admin message from step 1.1
`CREATE EXTERNAL VOLUME` fails: path is not covered by an external location	The `abfss://` path is outside a registered external location	Check **Catalog → External data → External locations** and use a folder inside one
`CREATE EXTERNAL VOLUME` fails: path overlaps another table or volume	Each external path can back only one table or volume	Pick a new empty sub-folder
Auto Loader or `dbutils.fs.ls` says access denied on storage	Missing `READ FILES` or `WRITE FILES` on the external location	Ask the admin for both privileges
`dbutils.fs.cp` or `spark.conf.set` is blocked	Some Standard (shared) cluster setups restrict APIs	Use serverless or a Dedicated (single-user) cluster
JDBC: connection timed out or login failed	Azure SQL firewall or workspace network blocks your compute, or wrong credentials	Check the firewall rule, then `dbutils.secrets.list('capstone03')` and the secret values
`Secret does not exist`	Scope or key name mismatch	List the scope; names are case-sensitive
`CREATE VOLUME` or `CREATE TABLE` denied	Missing `CREATE VOLUME` or `CREATE TABLE` on the schema	Ask the admin, or ask them to create the schema and grant you rights

13. Day 2 wrap-up

More decision-log rows

ID	Decision	Why
D13	`silver_flight_events` is rebuilt from Bronze on every run	Simple, repeatable; fine at this size
D14	`silver_flight_current` is seeded insert-only; status changes arrive only through the guarded MERGE	Rebuilding events can never erase applied CDC
D15	`silver_status_history` includes the original status from the flight event as sequence 0	The as-of join needs a state before the first change
D16	The as-of join returns the reported status for all 11,500 flights because every change is dated after the last event; the logic is proven by test U10	Report honestly what the data can and cannot show

Day 2 evidence checklist

- [ ] `docs/dq_rule_catalogue.md` (section 7 table, with actual counts filled in)
- [ ] Before/after table: 11,540 → 11,500 + 40, and the `Bronze = Silver + rejected` check
- [ ] Rejected-records output (40 × R10) and the `FLIGHT-0000501` trace
- [ ] `silver_status_history` (12,020 rows) and the `FLIGHT-0000001` trace
- [ ] Current-state status counts and the 11,500 uniqueness check
- [ ] Replay evidence: Silver rerun, CDC rerun with no change
- [ ] Unit test result table, all PASS
- [ ] Decision log D13 to D16 and the UTC/as-of limitation written in `docs/limitations.md`
- [ ] Notebooks `03` to `06` committed on `feature/day2-silver`, pull request opened and merged

What to be able to say out loud

If you can explain these five sentences without looking, you understand Days 1 and 2:

1. Bronze is an untouched raw copy with file name and load time on every row; Auto Loader's checkpoint makes re-runs safe.

2. Of 11,540 raw rows, 40 are exact duplicates; I keep the earliest copy, move the rest to a rejected table with a rule ID, and the numbers always add up.
3. The 35 extreme values are all in `fare_amount`; they are flagged, not deleted, because the spec says they are not automatically invalid.
4. Status changes are applied with a guarded MERGE that never lets an old change overwrite a newer one, and every change is kept in history.
5. The time zone column is unusable and there is no schedule data, so I documented my assumptions instead of pretending.

What comes next (Day 3 preview)

Gold reads `silver_flight_current` (one row per flight, with the post-CDC status) and joins to `bronze_airline_id` and `bronze_airport_id`. The cancellation rate will use `current_status = CANCELLED`, which is why Day 2's CDC step has to be right before you start Day 3. When you are ready, tell me and I will write Day 3 in the same format.